Federated Cascade Intrusion Detection for IIoT Networks: Poisoning-Resistant Aggregation, Connection-Level Evaluation and SDN Enforcement

Aaryan Prakash, Aaron B Ajay, Amal Roy, and Amogh Krishna J S

***Abstract*—IoT and IIoT sites rarely want to send raw traffic to a central detector, and a detector trained only on labelled attacks cannot recognise an attack type it has never seen. Federated learning keeps traffic local but lets any site send a poisoned update. We present FL-IDS: a LightGBM classifier names known attacks, a per-site autoencoder flags traffic unlike benign traffic, federated training uses a trust filter and trimmed-mean aggregation, and verdicts become OpenFlow 1.3 rules in an emulated network. We test the whole system on all 1,985,768 unique rows of Edge-IIoTset with every connection kept on one side of every split, after correcting three defects in the released data: a placeholder-spelling label leak (reported independently by Galal), column-shifted MITM rows, and a `tshark` version difference that zeroes a feature in live extraction. In six real multi-process runs the cascade detects 99.39 to 99.81% of known attacks (macro-recall). Packet false alarms are 2.5 to 16.3%, but 0.34 to 1.04% per benign connection. On attack types withheld from training the cascade detects 0.895 to 0.944 on average, against 0.723 to 0.757 for the classifier alone. With 30% sign-flipping sites the filter keeps the autoencoder working where plain averaging destroys it, but it admits attackers in 50 to 62% of their rounds and fails at 50%. Replaying all 14 attack captures ends in a verified drop rule each time, with 12 named correctly. Among 15 classifiers, tree models are interchangeable on the final 130 features (macro F1 0.979 to 0.985), while boosting leads on the 91 base features.**

***Index Terms*—Federated learning, intrusion detection, IIoT, Byzantine robustness, autoencoder, gradient boosting, software-defined networking, dataset leakage, Edge-IIoTset.**

I. Introduction

Machine-learning intrusion detection for IoT and industrial IoT (IIoT) networks usually assumes that traffic from every site can be pooled in one place. The operators of a plant floor, a hospital wing or a building network often cannot send packet data off-site, for privacy, contractual or bandwidth reasons. Federated learning (FL) [1] avoids the transfer: each site trains on its own traffic and shares only model parameters. FL-based intrusion detection is by now an established area with its own systematic review [2], so a federated detector on its own is not a contribution. A system that could be deployed still has to deal with five problems at once.

1. Known and unknown attacks. A supervised classifier names only the classes it was trained on. A new attack type is assigned to its nearest known class or to benign. An unsupervised detector flags departures from benign traffic, but cannot say what kind of attack it found.

The authors are with the Department of Computer Science & Engineering, BMS College of Engineering, Bengaluru, Karnataka, India.

2. Poisoned updates. Any site in a federation can send any update it likes. Byzantine-robust aggregation [3]–[5] limits the damage, but its performance drops when site data are non-IID [6], which is the normal case for IoT, because honest heterogeneous sites and attackers both look like outliers.

3. Enforcement. A detection that nobody acts on does not stop the attack. Software-defined networking (SDN) [7] provides a programmable point of enforcement.

4. Data integrity. Results on a public dataset are only as sound as its released files. Preprocessing artifacts that encode the label make published numbers unreliable [8]–[10].

5. Evaluation scale and leakage. Packets of one connection share much of their information. A test set that contains packets from connections also seen in training rewards memorising connections, and evaluations on small subsamples hide how a federated system behaves with realistic data volumes.

This paper describes FL-IDS, a system built to address all five in one pipeline, and reports what each part contributes when measured on its own. A LightGBM [11] classifier labels known attack types first. Traffic it labels benign, or labels as an attack with low confidence, passes to an autoencoder at the site, whose reconstruction error flags anything unlike benign traffic. This ordering follows a centralised two-stage design by Zhang et al. [12]. Here the autoencoder is trained federatedly with Flower [13], and a server-side trust filter screens the sites' updates before a trimmed-mean aggregation. The combined verdict drives OpenFlow 1.3 flow rules in an emulated network. The choice of boosting for the first stage rests on a model-selection study that ran in three earlier stages of the project and was repeated on Edge-IIoTset for this paper (Section VII).

Contributions

- **An integrated federated IDS** that combines a boosting-first detection cascade, a federated autoencoder, an update-level trust filter and SDN enforcement. Section II compares it with the closest published systems; each covers some of these parts, and none we found covers all of them.
- **An evaluation of the whole system on all 1,985,768 unique rows of Edge-IIoTset** [14], with every connection kept whole on one side of every split. It includes six real multi-process training runs (10 and 20 sites, three seeds), an ablation and a poisoning sweep over three seeds, a leave-one-class-out test for all 14 attack types

over three seeds, and replays of all 14 attack captures through the SDN path.
- **A measurement of how false alarms should be counted.** Counted per packet, false alarms are dominated by a few very long benign connections. Counted per connection, the rate is between 5.7 and 38 times lower.
- **Three corrections to Edge-IIoTset.** A placeholder-spelling leak that lets one feature separate benign from attack traffic perfectly (found independently of Galal [8], who reports the same mechanism), a set of MITM rows whose values sit under the wrong column names, and a `tshark` version difference that silently zeroes a feature in live extraction. To our knowledge the last two have not been reported.
- **A feature study** showing that the dataset's 91 standard features cap any classifier that sees only them at 0.969 macro recall, and that adding a service port, numeric payload-content features and connection-level aggregates lifts LightGBM to 0.983–0.985 macro F1 on connections it never saw in training.
- **A model-selection study** in three stages: two earlier stages on other datasets (MQTTset, DataSense) and a broad comparison of tree ensembles, linear, instance-based and neural classifiers on Edge-IIoTset, which gives the evidence for choosing boosting as the first stage.
- **Tests of the trust filter** against a white-box attacker that knows its rules, of pruning the autoencoder during and after training, and of a time-ordered split.

Section II reviews related work. Section III covers the dataset, its defects and the feature set. Section IV presents the system and Section V the experimental setup. Section VI reports the system results and Section VII the model-selection study. Section VIII discusses the results and lists the limitations, and Section IX concludes.

II. Related Work

A. Federated intrusion and anomaly detection

Hernández-Ramos et al. [2] review FL-enabled IDS work from 2018 to 2022 and classify it by model, dataset, aggregation function and library. Mothukuri et al. [15] run federated training rounds on GRU models for IoT anomaly detection. DeepFed [16] trains a combined CNN and GRU classifier across industrial cyber-physical sites. Neither includes a supervised stage plus an unsupervised stage for traffic outside the trained label set.

Autoencoders appear in several federated designs. Rey et al. [17] train both a supervised MLP and an unsupervised autoencoder across IoT devices for malware detection. They also show that plain averaging is badly exposed to malicious participants and test alternative aggregation functions as countermeasures. FedMSE [18] pairs a shrink autoencoder with a centroid one-class classifier and weights each local model by its error during aggregation. Kitsune [19] showed earlier that an ensemble of autoencoders can detect network intrusions online without supervision. The closest system to ours on the FL side is TA-FHIDF [20]. It combines an autoencoder, a 1D-CNN and a BiLSTM at the edge, scores client reliability by cosine similarity before global aggregation, and evaluates on UNSW-NB15, CICIDS2017 and Edge-IIoTset. As described in its abstract, it has no supervised-first cascade and does not mention network enforcement.

B. Byzantine-robust aggregation

Krum [3] chooses the update closest to its neighbours. Coordinate-wise median and trimmed mean [4] bound the influence of any single client on each coordinate, but only up to their breakdown point: a trimmed mean that cuts k updates from each side cannot absorb more than k colluding attackers. Fang et al. [21] showed that attackers who know the aggregation rule can craft updates that degrade these defences.

Direction-based defences compare updates by angle. FLTrust [5] holds a small clean root dataset on the server, computes a reference update from it, and weights each client by the ReLU-clipped cosine similarity of its update to that reference, so updates pointing away from the reference get no weight. SignGuard [22] derives magnitude, sign and similarity statistics from the received gradients and runs several filters on them before aggregation. Li et al. [23] exclude faulty updates through their geometric properties in parameter space and use momentum-based speculation. FoolsGold [24] identifies sybil clients from the diversity of their updates. Zhu et al. [25] compare angles at the level of convolutional kernels instead of whole models. In the IDS setting, pFL-IDS [26] analyses last-layer parameters, measures their angle to the centroid of the non-poisoned clients and re-weights clients with that score.

Non-IID data is the known weakness of these methods. In a systematic comparison, Li, Ngai and Voigt [6] found that the performance of all the aggregation schemes they tested drops significantly on non-IID data, and that some fail even when no Byzantine client is present. Our filter inherits this problem, and Section VI measures its effect.

C. Two-stage detection

Zhang et al. [12] propose the cascade our detection stage follows. A LightGBM classifier with focal loss makes a first decision after recursive feature elimination. For samples it calls normal, a residual from an autoencoder makes a second decision. Their evaluation is on NSL-KDD and UNSW-NB15. We keep the ordering and move the autoencoder into a federation. Our boosting model is trained on a server-held labelled set and broadcast to the sites. Federated boosting schemes such as SecureBoost [27] would remove that central labelled set, at a cost in protocol complexity that we did not take on here.

D. Federated detection with SDN mitigation

FedDICE [28] integrates FL with an SDN security architecture to detect ransomware spread across distributed clinical environments and uses SDN to remove infected devices. MiTFed [29] combines FL, blockchain and SDN, with an Ethereum-based scheme to maintain collaboration. Both establish the FL-decides, SDN-enforces structure we use. Neither includes a supervised-first cascade or an update-level poisoning filter.

E. Dataset integrity

Sommer and Paxson [30] argued that machine learning for network intrusion detection faces problems specific to the domain, among them evaluating a detector soundly. Arp et al. [9] list data snooping and spurious correlations among the common pitfalls of machine learning in security. Engelen et al. [10] found problems with traffic generation, flow construction, feature extraction and labelling in CICIDS2017 [31], reconstructed or relabelled more than 20% of its traffic traces, and regenerated the dataset from its packet captures, which is also how we repair the MITM rows (Section III-B). Outside security, Breck et al. [32] treat the gap between training data and the data a deployed model sees as a central problem for data validation. Phase B of our system checks for it directly (Section IV-F). For Edge-IIoTset in particular, Galal [8] shows that the benign and attack captures were serialised differently: a missing field is the string "0" in the normal-traffic branch and "0.0" in the attack branch, which lets several categorical columns recover the label on their own. Six standard classifiers score 1.0000 accuracy (the sixth 0.99998) on the curated ML subset under the authors' recipe, and the same audit repeated on the larger DNN file finds the artifact there too. Galal then rebuilds a benchmark from the raw captures, and runs FedAvg on that. Section III-B describes the same leak as we found it, and two further defects that, as far as we know, are not reported elsewhere.

Table I summarises the comparison. Each component exists in prior work, and several systems combine two or three of them. Our searches did not find a system that combines all of them. These searches were targeted rather than systematic, so we present the combination as an integration contribution and do not claim it is the first.

III. Dataset, Defects and Features

A. Edge-IIoTset

Edge-IIoTset [14] was captured on a physical IoT and IIoT testbed, and its authors position it for both centralised and federated learning. It ships in two forms, and we use both. The first is `DNN-EdgeIIoT-dataset.csv`, 2,219,201 packet rows with a selection of `tshark` fields prepared by the authors and a label, `Attack_type`, with 15 values (Normal and 14 attack types). All training and evaluation in this paper uses it. The second is the set of raw packet captures, one per device or attack. We replay the 14 attack captures through the emulated network (Phase B, Section IV-F).

We follow the authors' preprocessing recipe where it does not conflict with the corrections below. Of the CSV's 63 columns we keep 47 as base fields. The other 16 are identifiers, timestamps, raw payload and port fields, and `Attack_label`, which is a function of `Attack_type`. The seven nominal columns the recipe names are one-hot encoded, which gives 91 base features.

B. Defect 1: placeholder spelling encodes the label

Where a protocol field does not apply to a packet, the CSV holds a placeholder. That placeholder is spelled `0` in some source captures and `0.0` in others, and the spelling follows the label. In `mqtt.topic`, `mqtt.conack.flags`, `mqtt.protoname` and `dns.qry.name.len`, every Normal row holds `0` and every attack row holds `0.0`, with no overlap across 2.2 million rows. Each of these columns alone separates benign from attack traffic with an AUROC of 0.9998 or higher (Table II). A model trained on the recipe can therefore learn which capture script wrote a row instead of anything about the traffic.

We noticed the problem when an early leave-one-class-out run gave the autoencoder a detection rate of exactly 100% on every held-out class, which is not plausible for reconstruction error. We rewrite both spellings to `0` in all seven nominal columns before encoding and before deduplication, so that rows differing only in spelling are merged. On the final 130-feature matrix, the strongest single feature is the connection's RST count at an AUROC of 0.879 (Table II). The top six are all connection or port summaries, which describe behaviour rather than file format. Galal [8] traces the same artifact to the dataset's construction, in which benign and attack captures were parsed separately and the two branches wrote absent fields differently. We found the leak independently but after that report, and we do not claim it as new.

C. Defect 2: column-shifted MITM rows

The rows for the MITM attack (ARP spoofing with DNS) have values under the wrong column names. We re-extracted the authors' own MITM capture with their field list and aligned the result row by row with the released CSV. The CSV's `tcp.seq` column matches our `udp.time_delta` with agreement 1.000. Its `udp.time_delta` matches `dns.qry.type` (0.993), and its `dns.qry.qu` matches `dns.retransmit_request_in` (0.993). A model trained on these rows learns an ARP spoofing attack from a feature vector with no meaningful ARP or TCP values. We replace the 1,214 released MITM rows with 1,229 rows re-extracted from the capture using the same field list and encoding, which leaves 2,219,216 rows. After the repair, MITM traffic extracted live from the replayed capture matches the training rows exactly. The Modbus device capture has the same fault, but none of its rows are in the CSV, so live Modbus traffic is simply out of distribution.

D. Defect 3: `tshark` version drift

Phase B extracts features from replayed traffic with `tshark` and the authors' field list. Recent `tshark` releases (4.x) print booleans as `True`/`False` where the authors' output has `1`/`0`, and print hex values in short form (`0x00` instead of `0x00000000`). Left unhandled, this silently zeroes `tcp.flags.ack` on every live packet. Among the base features, that was the most informative single feature once the leak was removed. We map both forms back to the authors' spelling. Before the mapping, 6.6% of the base-feature rows extracted live from the Backdoor capture also occurred among the authors' Backdoor rows. After it, 99.8% did.

None of the three defects shows up in a test split drawn from the same CSV. The first two inflate or distort any model trained

TABLE I
COMPONENTS COVERED BY THE CLOSEST PUBLISHED SYSTEMS. A BLANK CELL MEANS THE COMPONENT IS NOT PART OF THE SYSTEM AS ITS AUTHORS DESCRIBE IT IN THE SOURCES WE REVIEWED.

System	FL	Unsup. AE	Supervised-first cascade	Update-level poisoning defence	SDN enforcement	Edge-IIoTset leak corrected
Mothukuri et al. [15]	✓					
DeepFed [16]	✓					
Rey et al. [17]	✓	✓		✓		
FedMSE [18]	✓	✓				
pFL-IDS [26]	✓			✓		
TA-FHIDF [20]	✓	✓		✓		
Zhang et al. [12]		✓	✓			
FedDICE [28]	✓				✓	
MiTFed [29]	✓				✓	
Galal [8]	✓					✓
FL-IDS (this work)	✓	✓	✓	✓	✓	✓

TABLE II
SINGLE-FEATURE AUROC FOR SEPARATING ATTACK FROM BENIGN ROWS, BEFORE THE PLACEHOLDER FIX (RAW COLUMNS OF THE RELEASED CSV) AND AFTER IT (THE SIX STRONGEST FEATURES OF THE FINAL 130-FEATURE MATRIX, 300,000-ROW SAMPLE).

Feature	Data	AUROC
`mqtt.topic = 0.0`	released CSV	1.0000
`mqtt.conack.flags = 0.0`	released CSV	1.0000
`mqtt.protoname = 0.0`	released CSV	1.0000
`dns.qry.name.len = 0.0`	released CSV	0.9998
`f_rst`	130-feature matrix	0.8789
`f_npkts`	130-feature matrix	0.8657
`svc_port`	130-feature matrix	0.8350
`f_fin`	130-feature matrix	0.8039
`f_pl_nonzero`	130-feature matrix	0.7328
`f_syn`	130-feature matrix	0.7108

on the released file. The third affects anyone who deploys such a model on live traffic with a current `tshark`. A fourth problem concerns only the captures: the released Vulnerability-scanner capture is damaged, with a record that claims a packet of 136 MB. Our extraction stops there and uses the 265,827 readable packets before the damage.

E. What the base features cannot separate

Correcting the defects does not make the 91 base features sufficient. In the SQL_injection, XSS, Uploading and Password captures, only 5 to 17% of packets carry any HTTP content. The rest are content-free TCP handshake and acknowledgement packets to port 80 on the same web server, and they look the same whichever attack produced them. We measured the effect by grouping rows by their exact 91-feature vector and giving each group its majority label, which is the best any classifier using these features can do. Of all rows, 2.31% share a feature vector with a row of a different class, which caps accuracy at 0.9875 and macro recall at 0.9692. The separating information is in columns the recipe drops: the payload and URI text, and the ports.

F. Feature set

We add 39 features to the 91, giving the 130 that every model in this paper uses (except the base-feature comparisons in Sections VI-G and VII). IP addresses, absolute times, ephemeral ports and row order are never features, because each of them identifies the capture file and through it the label. They are used only to group packets into connections.

- **Service port** (1 feature): the server side of the connection, taken as the smallest TCP or UDP port below 32,768, or -1 for ICMP and ARP. Client ports in the ephemeral range are excluded because their values follow capture time.
- **Content** (18 features), computed as numbers from three dropped columns, the TCP payload, the full request URI and the HTTP file data: decoded payload length, byte entropy and printable share; an HTTP request flag and the response status code; header and X-header counts; URI length, parameter count and percent-encoding count; file-data length; a count of special characters; and match counts for six keyword groups (SQL syntax, script injection, file upload, login terms, path traversal and command execution, and scanner tool names).
- **Connection** (20 features). Packets are grouped by IP pair, protocol and port pair, in capture order. A new connection starts after 60 s of idle time or 120 s of activity. The 2,219,216 packets form 392,782 connections. Every packet then carries its connection's packet count, duration and packet rate; SYN, FIN and RST counts; payload sum, maximum and non-empty share; HTTP request count; the maximum status code, X-header count, entropy and URI length; and keyword totals.

Connection features summarise a whole connection record of up to 120 s, as CICFlowMeter-style datasets do [31]. A packet's features therefore include information from later packets of the same connection, so the system labels connection records rather than deciding on each packet as it arrives. The same fact is why we split by connection (Section V-A): with a random row split, packets of one connection fall on both sides and share most of their features.

Exact duplicates are removed after the features are computed, leaving 1,985,768 rows in 342,814 connections. Table III gives the class distribution. Normal traffic is 72.3% of the rows. MITM (406 rows, all in a single connection) and Fingerprinting (872 rows) together are under 0.1%.

TABLE III
CLASSES OF EDGE-IIOTSET. "RELEASED" COUNTS ROWS OF THE RELEASED CSV. "USED" COUNTS UNIQUE ROWS AFTER THE CORRECTIONS AND THE 130-FEATURE DEDUPLICATION; THE SHARE IS OF THE USED TOTAL.

Class	Released	Used	%
Normal	1,615,643	1,436,597	72.34
DDoS_UDP	121,568	121,567	6.12
DDoS_ICMP	116,436	68,758	3.46
SQL_injection	51,203	50,981	2.57
DDoS_TCP	50,062	50,062	2.52
Vulnerability_scanner	50,110	50,044	2.52
Password	50,153	50,033	2.52
DDoS_HTTP	49,911	49,768	2.51
Uploading	37,634	37,243	1.88
Backdoor	24,862	24,167	1.22
Port_Scanning	22,564	20,213	1.02
XSS	15,915	15,172	0.76
Ransomware	10,925	9,885	0.50
Fingerprinting	1,001	872	0.04
MITM	1,214	406	0.02
Total	2,219,201	1,985,768	

IV. SYSTEM DESIGN

A. Overview and threat model

FL-IDS runs in two phases, kept separate on purpose so that model quality and network behaviour can be evaluated and debugged independently (Fig. 1). Phase A is offline federated training. It produces a boosting classifier, a global autoencoder and, for each client, a feature scaler and an anomaly threshold, all saved as one self-describing bundle. All detection and robustness results in this paper come from Phase A. Phase B loads that bundle into an emulated SDN, replays real captures, and turns the cascade's output into flow rules.

The federation has a server and n clients. Each client stands for one site and holds only that site's traffic. Client data are non-IID. The server is trusted, and it holds a small labelled calibration set (Section IV-B). A fraction f of clients is malicious. A malicious client receives the global model like any other, trains on its own data, and then sends the server an arbitrary update. It may also misreport its number of training examples, which matters for example-weighted rules such as FedAvg. We evaluate two concrete attackers: a sign-flipping client, and a white-box client that knows the filter's rules and shapes its update to pass them (Section V-C). Data-poisoning clients that send honestly computed updates on manipulated local data are outside what an update-level filter can detect; we discuss them in Section VIII.

B. Boosting stage

The first stage is a LightGBM multiclass classifier over all 15 labels. Its training labels come from a server-held calibration set of about 5% of all rows, chosen as whole connections, class by class (Section V-A). Part of this set is held back to score each version of the booster: a fifth by configuration, 20 to 28% in practice because whole connections are taken. The booster weights each class by $\sqrt{N_{\max}/N_c}$, where N_c is the class's training count, so that rare classes count for more without the instability of full inverse-frequency weights. We chose server-held labels over client-held labels because a supervised model needs labels, and IoT sites in practice rarely have analyst-labelled attack traffic. The cost is that this one component is centrally trained (Section VIII). The server trains the classifier before round 1, so clients never filter with an untrained model. In each round, the serialised model (about 9 to 10 MB) travels to the clients inside the round's fit configuration. It is not averaged like the autoencoder; every client runs the same copy locally and caches its filtering result until the model changes.

The boosting stage reads raw feature values. The autoencoder reads per-client normalised values (Section IV-C). We separated the two after finding that per-client normalisation destroys a shared tree model. Under strong label skew, a client's own scaler can shift a feature by orders of magnitude, so the split thresholds learned on the calibration set no longer apply. In an early test, the same held-out rows passed through a heavily skewed client's scaler were classified almost entirely wrongly.

Incremental update. Every three rounds, each client that passed the trust filter in that round surfaces up to 50 rows its autoencoder flagged, with labels confirmed by an analyst. False alarms come back labelled Normal. The server continues training the booster on the calibration set plus all alerts so far (50 more boosting rounds, via LightGBM's initial-model option). If the alerts contain a class the booster has never seen, the server retrains it from scratch, because continued training cannot learn a new class: its predicted probability is near 10^{-14}, the softmax Hessian for it is near zero, and the minimum-Hessian constraint then blocks every split. This mechanism relaxes the rule that raw traffic never leaves a client. Up to 50 confirmed alert rows per surviving client per update round are sent to the server. We report it as a design trade-off.

C. Client autoencoder

Each client trains a dense autoencoder with layers $130 \to 32 \to 16 \to 8 \to 16 \to 32 \to 130$, ReLU between layers and a linear output. It has 9,834 parameters, of which 9,600 are weights and 234 are biases. Training minimises mean squared reconstruction error with Adam [33] (learning rate 10^{-3}, batch size 64, five local epochs per round). A client trains only on the local rows that the current boosting model passes to the second stage: rows predicted Normal, or predicted as an attack with probability below the cascade threshold. The autoencoder therefore models traffic that the supervised stage considers benign or is unsure about.

Inputs are z-scored with each client's own scaler. Global normalisation would leak cross-client distribution statistics and remove the heterogeneity we want to test. We clip z-scores at ± 10. Counter-like fields such as `udp.stream` and `tcp.seq` can barely vary inside one client's shard. Unclipped, ordinary rows from another part of the capture can then land thousands of standard deviations out and dominate that client's reconstruction error.

Each client calibrates its anomaly threshold θ_c as the 97th percentile of reconstruction error on a held-out benign valida-

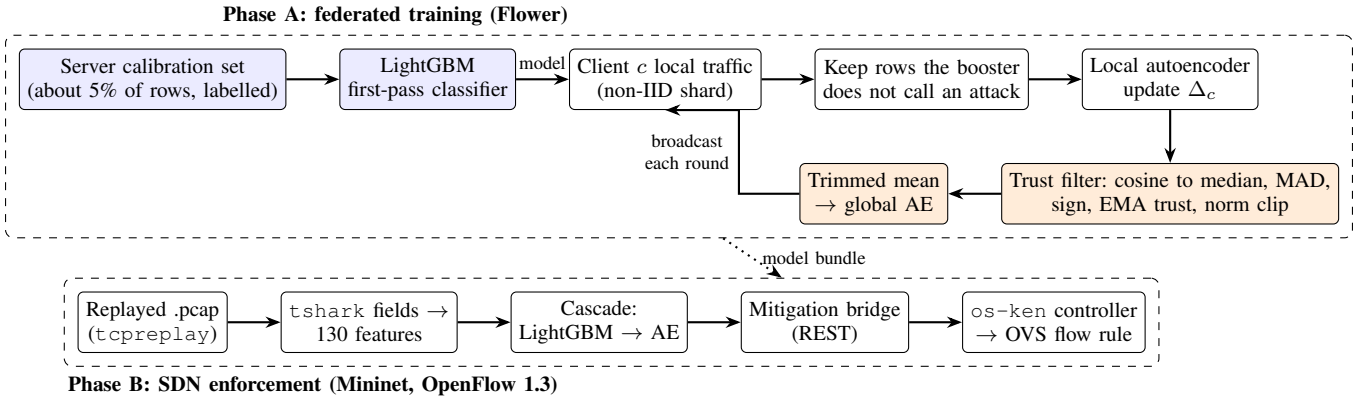


Fig. 1. FL-IDS architecture. Phase A trains the autoencoder federatedly on traffic the boosting stage passes, with a trust filter in front of aggregation. Phase B loads the resulting bundle and enforces its verdicts as OpenFlow rules.

tion slice of its own data. It recomputes the threshold every round, because the global model changes every round. By construction, about 3% of a client's benign validation traffic lies above its threshold. A client whose shard has no benign validation rows gets $\theta_c = \infty$, so its autoencoder never flags. With each update, a client also returns a 50-bin histogram of its reconstruction errors over $[0, 10]$ for monitoring.

D. Cascade decision rule

Evaluation, the dashboard and the SDN bridge all use one implementation of the cascade. For a row x at client c, let $\hat{y}$ be the booster's predicted class and $\hat{p}$ its probability, and let $e(x)$ be the autoencoder's reconstruction error on the client-normalised row. With confidence threshold $\gamma = 0.7$,

$$\text{output}(x) = \begin{cases} (\hat{y}, \hat{p}) & \text{if } \hat{y} \neq \text{Normal and } \hat{p} \geq \gamma, \\ \big(\text{anomalous},\ \text{clip}_{[0,1]}\big(e(x)/\theta_c - 1\big)\big) & \text{else if } e(x) > \theta_c, \\ (\text{benign}, \cdot) & \text{otherwise.} \end{cases} \tag{1}$$

The second stage outputs *anomalous*, not an attack type. A reconstruction error can say that a row does not look like benign traffic, but not what attack it belongs to.

E. Trust filter and aggregation

Let w_t be the global autoencoder parameters sent out at the start of round t, and w_i the parameters client i returns. The filter works on deltas $\Delta_i = w_i - w_t$. Raw weights are dominated by the shared global model, so the cosine similarity between any two clients' raw weights is close to 1 whatever their behaviour. As a reference direction the server uses the coordinate-wise median $r = \text{med}(\Delta_1, \ldots, \Delta_n)$, and for each client it computes $s_i = \cos(\Delta_i, r)$. Client i is excluded from the round if any of three conditions holds:

1) **MAD outlier:** $s_i < m - 3d$, where m is the median of the round's similarities and d their median absolute deviation [34];
2) **opposes consensus:** $s_i < 0$;
3) **low trust:** $\tau_i < 0.5$ after this round's update, where $\tau_i \leftarrow \alpha\,(s_i + 1)/2 + (1 - \alpha)\tau_i$ with $\alpha = 0.3$ and $\tau_i = 1$ initially.

Algorithm 1 One round of trust-filtered aggregation

Require: global weights w_t; client weights $w_1, \ldots, w_n$; trust scores τ

$\Delta_i \leftarrow w_i - w_t$ for all i
$r \leftarrow$ coordinate-wise median of $\{\Delta_i\}$
$s_i \leftarrow \cos(\Delta_i, r)$; $m \leftarrow \text{med}(s)$; $d \leftarrow \text{med}(|s - m|)$
$\tau_i \leftarrow 0.3\,(s_i + 1)/2 + 0.7\,\tau_i$
$S \leftarrow \{i : s_i \geq m - 3d \ \wedge\ s_i \geq 0 \ \wedge\ \tau_i \geq 0.5\}$
if $S = \emptyset$ **then**
  return w_t
end if
$c \leftarrow 2 \cdot \text{med}_j \, \|\Delta_j\|$
$\tilde{\Delta}_i \leftarrow \Delta_i \cdot \min(1,\, c/\|\Delta_i\|)$ for $i \in S$
$k \leftarrow \lfloor 0.15\,|S| \rfloor$
return $w_t + \text{TrimmedMean}_k(\{\tilde{\Delta}_i\}_{i \in S})$

Each rule addresses a specific failure. The MAD threshold is relative to the round: under heavy non-IID skew, honest clients can have low similarity to the median, and a fixed cosine cutoff would either reject them or admit attackers depending on the split. MAD alone is not enough, though. In an earlier version that used only this rule, honest heterogeneity widened the MAD band until attackers with negative similarity fell inside it. The sign rule is the same boundary FLTrust uses when it gives updates with negative cosine to the reference zero weight [5]. It is a fixed point, the sign, rather than a tuned value. The trust rule makes a client's history count, the role temporal analysis plays in [23].

Cosine similarity ignores magnitude. The filter also clips each surviving delta to $2\times$ the round's median norm, $\tilde{\Delta}_i = \Delta_i \min\big(1,\, 2\,\text{med}_j \|\Delta_j\| / \|\Delta_i\|\big)$. The clipped survivors are combined by a coordinate-wise trimmed mean [4] that drops $\lfloor 0.15|S| \rfloor$ values from each side, where S is the survivor set. The result is added to w_t. If no client survives, the round's aggregation is skipped. Example counts reported by clients are ignored throughout. Algorithm 1 summarises one round. The filter keeps no state other than the trust scores, and it never sees data. The aggregation runs inside a custom Flower strategy that subclasses FedAvg. Its fit-configuration step records w_t for the

delta computation and attaches the current boosting model. Its aggregation step runs Algorithm 1 and passes the surviving clients' alerts to the boosting update.

F. SDN enforcement

Phase B rebuilds every model from the Phase A bundle, never from the current configuration. The bundle holds the booster, the final autoencoder weights, each client's scaler and threshold, the z-score clip, the cascade settings, held-out metrics and provenance (seed, commit, time). Phase B rejects a bundle whose thresholds, recomputed from the saved weights, do not match the clients' final-round values.

Mininet [35] brings up one Open vSwitch [36] switch and one host per FL client. Host h_{i+1} is scored with client i's scaler and threshold. The switch is controlled by an OpenFlow 1.3 controller we built on `os-ken`, the maintained OpenStack fork of Ryu. A real Edge-IIoTset capture is replayed from a host with `tcpreplay`. `tshark` extracts the authors' fields with the corrections of Section III-B, and the same code that built the training matrix derives the 130 features, including the connection features, from the capture. To check for training/serving skew [32], we compared the live rows from the Backdoor capture with that capture's training rows: 99.8% of the 24,914 live rows are identical to a training row across all 130 features. The cascade then classifies every row.

The rows are reduced to one verdict per device. If any row is non-benign, the verdict is the most frequent non-benign label, its confidence is the mean confidence of those rows, and its stage is the stage that made most of those calls. Otherwise the verdict is benign. A REST bridge receives {device, label, confidence, stage} and asks the controller for an action. Confidence of at least 0.85 installs a drop rule on the device's source address. Confidence of at least 0.5 attaches an OpenFlow meter that rate-limits it. Anything lower is left to monitoring. The controller reads switch state back through OpenFlow multipart statistics requests, so what it reports is what `ovs-ofctl dump-flows` shows. The topology is written so a switch port can bridge to a physical or virtual NIC, which would put an external device's traffic under the same controller. That path is designed but not yet built.

V. Experimental Setup

A. Splits by connection

Every split in the federated system keeps each connection whole. The server's calibration set takes whole connections, class by class, until it holds about 5% of each class's rows. The remaining pool, all of it, is divided among the clients with a Dirichlet label-skew partition [37] with concentration $\alpha = 0.3$. For each class, a Dirichlet draw gives each client a target share, and connections in random order go to whichever client is furthest below its target. Each client then sets aside about 15% of its rows as a test slice and about 15% of its remaining benign rows as a benign validation slice for threshold calibration, again by whole connections. The rest is its training data. No connection appears in more than one of these sets.

TABLE IV
Data sizes and Phase A wall time of the six real runs. "Server train" and "held back" are the two parts of the calibration set; "no benign val." counts clients with no benign validation rows.

Clients	Seed	Server train	Held back	Client rows	Test rows	No ben. val.	Phase A (s)
10	42	79,866	20,315	1,885,587	288,416	1	1844
10	43	72,803	27,221	1,885,744	283,090	0	1397
10	44	71,853	28,242	1,885,673	288,366	2	1754
20	42	79,866	20,315	1,885,587	283,427	3	1260
20	43	72,803	27,221	1,885,744	292,222	2	1063
20	44	71,853	28,242	1,885,673	295,566	0	1225

A class with fewer than 20 connections cannot be split this way. Only MITM is affected: its 406 rows form a single connection, so all of it goes to the server's calibration set. The booster learns MITM, but no client holds MITM rows and no test slice contains any. MITM is evaluated only by the zero-day experiment and the Phase B replay. Twenty-four connections (655 rows) carry two labels and are split by label.

Table IV gives the resulting sizes. The seed changes the calibration set, the partition and the model initialisation. Because whole connections are chosen, the server set varies between 100,024 and 100,181 rows across seeds. Of it, 71,853 to 79,866 rows train the booster and the rest score it. The clients together hold about 1.89 million rows, and their test slices hold 283,090 to 295,566 rows. Strong skew leaves some clients without benign validation rows. They get an infinite threshold, so their autoencoder never flags.

B. Real runs and in-process experiments

Six real runs train the system as a deployed federation would: a Flower server and one operating-system process per client, 20 rounds, no attackers, with 10 and 20 clients and seeds 42, 43 and 44. Each run saves a model bundle, which we score on the clients' test slices. The 10-client, seed-42 bundle also drives Phase B.

The ablation, poisoning sweep, zero-day, stealth-attacker, pruning and clip experiments take separate training jobs that drive the same client, attacker and aggregation code in one process instead of over gRPC, with 10 clients and 20 rounds on the same full-pool splits. The sweep, ablation, zero-day, stealth-attacker, pruning-during-training and clip experiments all run with three seeds. Pruning after training is a single run on the seed-42 bundle.

C. Attackers

A *sign-flipping* client trains honestly for the round and then returns $w_t - 5(w_i - w_t)$, its true delta negated and amplified five times. It reports the largest example count of any client, so that FedAvg, which weights updates by reported counts, gives it as much weight as possible. The trust filter and trimmed mean ignore counts. At a malicious fraction f, $\mathrm{round}(fn)$ clients are drawn without replacement using the run's seed.

A *stealth* client is a white-box attacker that knows the filter's cosine rule and norm clip. It estimates the consensus direction as the last aggregate update it can observe (this round's global weights minus last round's). Its update keeps a cosine of 0.1 to

that direction and spends the rest of its norm budget, 1.5 times its honest norm and so below the $2\times$ clip, on the negative of the part of its honest update that is orthogonal to the consensus. In round 1 it has no estimate and behaves honestly. This is one adaptive strategy, not a bound over all of them.

We do not use label flipping or input corruption as test attackers. The autoencoder never sees labels, and corrupted inputs mostly make a client look like one more heterogeneous client, so neither would test an update-level defence.

D. Variants

Every variant uses the same data, partition and seed.

- `full_pipeline`: the trust filter and trimmed mean, the boosting pre-filter for autoencoder training, and the cascade of Eq. (1).
- `plain_fedavg`: as above, with FedAvg in place of the filter and trimmed mean.
- `trimmed_mean_only`: the trimmed mean without the trust filter.
- `autoencoder_only`: the autoencoder trained on all local rows with no boosting pre-filter. Detection is the autoencoder alone.
- `boosting_only`: the cascade rule without the second stage, and no FL. A row counts as detected when the booster labels it an attack with probability at least γ.

E. Metrics

About 72% of the rows are benign, so accuracy mostly tracks the benign false-positive rate, and it even rises when a poisoned autoencoder stops flagging anything. We report the following instead.

- **Attack macro-recall:** the share of attack rows flagged (labelled with any attack type, or anomalous), averaged over the attack classes present in the test slices with equal weight. In the federated tests this is 13 classes, because MITM has no test rows.
- **Attack micro-recall:** the same, pooled over all attack rows.
- **Packet false-alarm rate:** the share of benign test rows flagged.
- **Connection false-alarm rate:** the share of benign test connections in which at least half the rows are flagged. We also report the median client's packet rate, and the share of all false-alarm packets that come from the ten largest benign connections.
- **Autoencoder-alone macro-recall:** attack macro-recall when the autoencoder flags every row by itself. It isolates the federated component, whose damage under poisoning the cascade otherwise hides.
- **Attacker survival:** the share of attacker-rounds in which an attacker's update passed the filter into aggregation.

The test set is the union of the clients' test slices, and each row is scored with its own client's scaler and threshold, as in deployment.

Zero-day protocol. For each of the 14 attack classes in turn, we remove that class from the calibration set and from every client's data before partitioning, so it is absent even from the scaler statistics. We train, then score every row of the held-out class (from 406 rows for MITM to 121,567 for DDoS_UDP) as unseen traffic. Each row is assigned to a random client and scored with that client's scaler and threshold. A row counts as detected if the cascade flags it in any way. We report the detection rate per class and its mean over the 14 classes, for the classifier alone, the autoencoder alone and the cascade.

Pruning. We prune the autoencoder by global weight magnitude, zeroing the smallest fraction of weight-matrix entries and keeping the biases. During training, the server re-prunes the global model after every aggregation, with the target sparsity ramped linearly over the first ten rounds. After training, we prune the final model of the 10-client, seed-42 real run and recalibrate the thresholds. Reported sizes assume a sparse encoding (float32 values plus int32 indices). Clients still train and send dense updates, so this shrinks the stored model, not the per-round communication.

F. Standalone classifier studies

Three studies evaluate the first-stage classifier alone and centrally.

- **Feature study.** LightGBM on all rows at the natural class distribution, for each step from the 91 base features to the full 130. Each feature set gets its own deduplication. Each is evaluated on two 80/20 splits: a split stratified by class over individual rows, and the first fold of a five-fold stratified group split with the connection as the group, so that no connection has rows on both sides. We treat the second as the main result. MITM lies wholly on one side of it and is left out of its macro F1. LightGBM uses 63 leaves, learning rate 0.1, feature and bagging fractions of 0.8, leaf regularisation, early stopping (patience 50) on 10% of the training rows and, in the final configuration, the class weights of Section IV-B. It ran with four threads, so it is reproducible only to within thread-scheduling noise.
- **Time-ordered split.** The same final configuration, with each class's earliest 80% of connections in training and its latest 20% in test, for seeds 42 to 44. CSV row order is capture order within each class. The CSV records time of day without a date, so this orders by row position, not by a verified clock.
- **Model selection.** LightGBM against other classifiers on the same split. Section VII describes this study in full.

G. Hyperparameters and implementation

Table V lists the settings. The cascade threshold, 97th percentile, trim fraction, norm-clip multiplier and SDN thresholds come from the project specification, set in an earlier prototype, and were not tuned on the test data. The LightGBM regularisation was added because training without it overfitted the small calibration set, and its benign false-alarm rate swung widely when a small share of calibration rows changed. In the pipeline, LightGBM runs single-threaded in deterministic mode, because multi-threaded histogram construction did not reproduce exactly across identical runs.

TABLE V
HYPERPARAMETERS OF THE FEDERATED SYSTEM (FROM THE RUN CONFIGURATION FILES).

Component	Parameter	Value
Data	clients n / Dirichlet α / seeds	10 or 20 / 0.3 / 42, 43, 44
	test / benign-validation fraction	0.15 / 0.15
	z-score clip / min. connections per class	$\pm$10 / 20
LightGBM	calibration fraction / held-back share	0.05 / 0.2
	boosting rounds / learning rate / leaves	300 / 0.05 / 31
	min. leaf Hessian / L2 / class weights	1.0 / 1.0 / square root
	update every / extra rounds / alerts per client	3 rounds / 50 / 50
Autoencoder	layers	130–32–16–8–16–32–130
	optimiser / lr / batch / local epochs	Adam / 10^{-3} / 64 / 5
	anomaly percentile	97
Cascade	confidence threshold γ	0.7
FL	rounds	20
Trust filter	MAD multiplier / min. cosine / min. trust	3.0 / 0.0 / 0.5
	EMA α / norm-clip multiplier / trim per side	0.3 / 2.0 / 0.15
Attackers	sign-flip amplification / claimed count	5 / largest client
	stealth cosine target / norm multiplier	0.1 / 1.5
SDN	block / rate-limit confidence	0.85 / 0.5

The system is written in Python with PyTorch [38] 2.14, Flower [13] 1.38, LightGBM 4.7 and `os-ken` 4.2, with Mininet and Open vSwitch for the network and `tcpreplay` and `tshark` for traffic. All experiments ran in an Ubuntu virtual machine with 20 virtual CPUs and 48 GB of memory. A real 20-client run needs about 28 GB, so real runs were never run alongside other jobs. The test suite (247 passed, one skipped) passes in this environment.

VI. SYSTEM RESULTS

A. Real multi-process runs

Table VI reports the six real runs on the clients' test slices, 283,090 to 295,566 rows per run, with no attackers. Known attacks are detected almost completely. The cascade's micro-recall is 0.9956 to 0.9989 and its macro-recall over the 13 attack classes present is 0.9939 to 0.9981. The booster alone reaches 0.9954 to 0.9981 micro-recall, so on known attacks the autoencoder adds between 0.0002 and 0.0008. The bootstrap booster, scored on the held-back part of the calibration set, reaches 0.9947 to 0.9980 accuracy and 0.9609 to 0.9871 macro F1 (Table VII).

False alarms. The packet false-alarm rate varies from 2.5% to 16.3% across the six runs, and nearly all of it comes from the second stage: the booster alone flags 0.012% to 0.046% of benign packets. The packet rate is a poor summary, because a few very long benign connections dominate it. In the 10-client, seed-42 run, client 9 holds a benign connection of 11,764 packets that the autoencoder flags in full. In five of the six runs, 80 to 94% of all false-alarm packets come from the ten largest benign connections, and 52% in the sixth. The median client's packet rate is 2.9 to 3.2% in every run, close to the 3% the 97th-percentile threshold implies. Counted per connection, 0.34 to 1.04% of the 10,765 to 12,519 benign test connections have half or more of their packets flagged (Fig. 2). An operator who acts on connections would see roughly one false alarm per hundred benign connections. One who acts on packets would see many more, concentrated in a handful of long-lived sessions.

The autoencoder on its own. Alone, the autoencoder detects between 22% and 72% of attack packets. It is weaker with 10 clients (0.220 to 0.463) than with 20 (0.619 to 0.718),

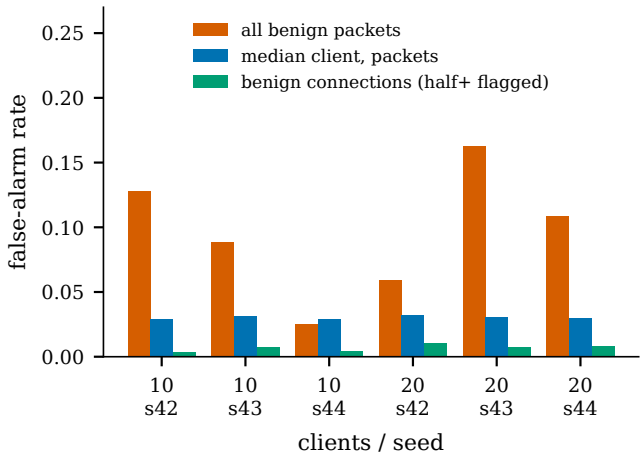


Fig. 2. False-alarm rate of the six real runs, counted three ways.

and it varies strongly with the seed. Table VIII shows why it still matters little for known attacks. Per class, the booster detects 0.984 to 1.000 and the cascade 0.987 to 1.000, while the autoencoder alone ranges from 0.037 on Backdoor to 0.939 on DDoS_TCP. Its value lies in attack types the booster has never seen (Section VI-C).

The filter with no attackers. Even with no attacker present, the filter excluded 6 to 12 of 200 client-rounds in the 10-client runs and 12 to 19 of 400 in the 20-client runs (Table VII). Every exclusion but one was a MAD outlier. These are honest clients whose non-IID updates differ from the median, which is the confound discussed in Section VIII.

Phase A took 1,063 to 1,844 s for 20 rounds (Table IV); 871 to 1,633 s of that is federated training, 16 s the calibration split, bootstrap classifier and client partition, and 156 to 191 s evaluation. Some runs shared the machine with other jobs, so these times are upper bounds.

B. Ablation and poisoning sweep

The in-process experiments reproduce the real runs closely. With no attackers, in-process attack macro-recall is 0.996, 0.996 and 0.994 for the three seeds, against 0.9958, 0.9947 and 0.9939 in the 10-client real runs. Table IX compares the five variants with three sign-flipping clients out of ten, and Table X with Fig. 3 follows the full pipeline from 0% to 50% attackers.

Known attacks are safe from poisoning. Every variant that includes the booster keeps attack macro-recall between 0.982 and 0.997, and the full pipeline stays between 0.985 and 0.997 at every attacker fraction up to 50%. The booster is trained on the server, so poisoned updates cannot reach it. Reported on its own, this number would hide what happens to the federated part.

Robust aggregation protects the autoencoder at 30%. The full pipeline's autoencoder detects 0.490, 0.373 and 0.245 at 30% attackers, against 0.457, 0.492 and 0.221 with no attackers. Under plain FedAvg it detects 0.013, 0.002 and 0.092, and under the trimmed mean without the filter 0.205, 0.000 and 0.025. The trimmed mean alone fails because, with

TABLE VI
SIX REAL MULTI-PROCESS RUNS (20 ROUNDS, NO ATTACKERS), SCORED ON THE CLIENTS' TEST SLICES. RECALL COLUMNS: CASCADE MACRO- AND MICRO-RECALL, BOOSTER-ALONE MICRO-RECALL, AUTOENCODER-ALONE MACRO-RECALL. FALSE-ALARM COLUMNS: ALL BENIGN TEST PACKETS, THE MEDIAN CLIENT'S PACKETS, BENIGN CONNECTIONS WITH HALF OR MORE OF THEIR PACKETS FLAGGED, AND THE SHARE OF FALSE-ALARM PACKETS THAT COME FROM THE TEN LARGEST BENIGN CONNECTIONS.

		Recall on known attacks				False alarms			
Clients	Seed	Macro	Micro	Booster	AE alone	Packets	Median client	Connections	Top-10 share
10	42	0.9958	0.9974	0.9970	0.461	0.1275	0.0286	0.0034	0.94
10	43	0.9947	0.9978	0.9971	0.463	0.0884	0.0313	0.0074	0.80
10	44	0.9939	0.9956	0.9954	0.220	0.0248	0.0293	0.0039	0.89
20	42	0.9981	0.9989	0.9981	0.718	0.0588	0.0322	0.0104	0.52
20	43	0.9970	0.9980	0.9974	0.619	0.1628	0.0307	0.0076	0.87
20	44	0.9972	0.9978	0.9972	0.709	0.1089	0.0296	0.0081	0.90

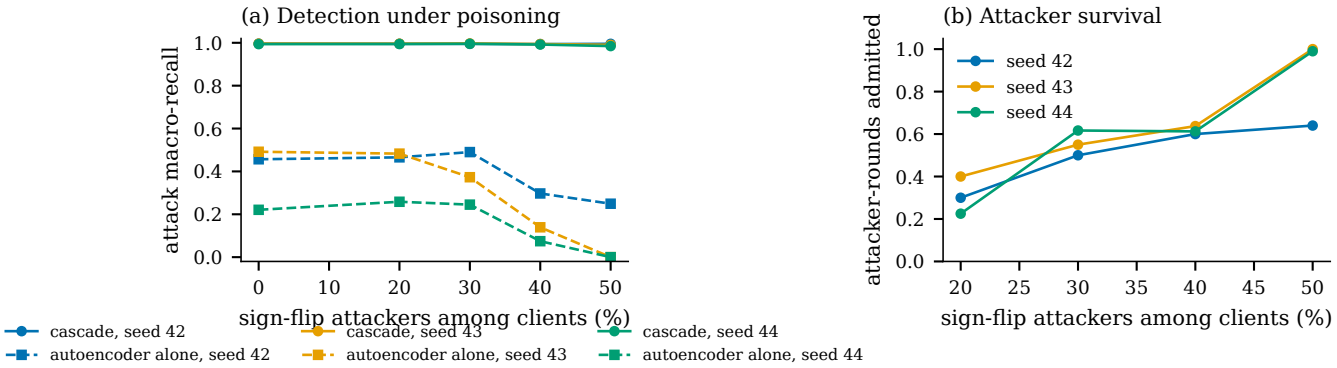


Fig. 3. Full pipeline under increasing shares of sign-flipping clients, three seeds. (a) Known-attack macro-recall of the cascade (solid) and macro-recall of the autoencoder alone (dashed). (b) Share of attacker-rounds in which an attacker passed the filter.

TABLE VII
BOOTSTRAP BOOSTER SCORED ON THE HELD-BACK CALIBRATION ROWS, AND HONEST CLIENT-ROUNDS THE TRUST FILTER EXCLUDED DURING EACH REAL RUN (NO ATTACKERS PRESENT).

Clients	Seed	Accuracy	Macro F1	Client-rounds excluded
10	42	0.9947	0.9808	12 / 200
10	43	0.9971	0.9609	6 / 200
10	44	0.9980	0.9871	7 / 200
20	42	0.9947	0.9808	19 / 400
20	43	0.9971	0.9609	12 / 400
20	44	0.9980	0.9871	12 / 400

TABLE VIII
RECALL PER ATTACK TYPE BY STAGE, 10 CLIENTS, SEED 42, WITH THE RANGE OF CASCADE RECALL OVER ALL SIX REAL RUNS. MITM HAS NO TEST ROWS (SECTION V-A).

Class	Test rows	Booster	AE alone	Cascade	Cascade, six runs
Backdoor	2,263	0.9885	0.037	0.9929	0.9729–0.9983
DDoS_HTTP	6,915	0.9899	0.481	0.9899	0.9856–0.9963
DDoS_ICMP	9,520	1.0000	0.317	1.0000	0.9997–1.0000
DDoS_TCP	6,717	1.0000	0.939	1.0000	1.0000–1.0000
DDoS_UDP	17,755	0.9999	0.306	1.0000	1.0000–1.0000
Fingerprinting	129	0.9845	0.070	0.9922	0.9730–1.0000
Password	7,795	0.9913	0.252	0.9914	0.9875–0.9963
Port_Scanning	2,929	1.0000	0.684	1.0000	0.9982–1.0000
Ransomware	691	0.9841	0.488	0.9870	0.9847–0.9975
SQL_injection	7,228	0.9990	0.894	0.9996	0.9976–1.0000
Uploading	5,567	0.9998	0.549	0.9998	0.9995–1.0000
Vuln. scanner	7,216	0.9946	0.501	0.9961	0.9925–0.9996
XSS	2,274	0.9960	0.471	0.9969	0.9893–0.9991

ten clients and a 15% trim, it removes one update from each side, and three colluding attackers outnumber that. This is the expected breakdown of the rule [4]. The margin over plain FedAvg is smallest at seed 44, where the autoencoder was weak to begin with.

The filter admits many attackers at this scale. Attackers reached aggregation in 23 to 40% of their rounds at 20% attackers and in 50 to 62% at 30%. The autoencoder still holds at 30%. Our reading, which we have not tested separately, is that the filter removes enough attackers for the stages after it to cope: at a survival of 0.50 to 0.62, about 1.5 to 1.9 of the three attackers pass in an average round, and the norm clip and one-per-side trim bound what they can do. At 40% the autoencoder degrades to 0.297, 0.139 and 0.074. At 50%, attackers are admitted in 64%, 100% and 99% of their rounds and the autoencoder fails in two of three seeds. Once half the clients attack, the coordinate-wise median moves toward their direction, and any defence that takes the median as its consensus fails there.

The pre-filter matters. Trained on all local traffic instead of boosting-filtered traffic, the autoencoder detects only 0.105, 0.073 and 0.051. Filtering out traffic the booster already recognises as attack leaves the autoencoder a much cleaner picture of benign behaviour.

The cascade's gain on known attacks is small and its false

TABLE IX
ABLATION WITH 30% SIGN-FLIPPING CLIENTS, 10 CLIENTS, 20 ROUNDS. EACH CELL LISTS SEEDS 42 / 43 / 44. THE FULL-PIPELINE ROW IS THE SWEEP'S 30% POINT.

Variant	Attack macro-recall	AE-alone macro-recall	Benign FPR (packets)
full_pipeline	0.995 / 0.997 / 0.995	0.490 / 0.373 / 0.245	0.1112 / 0.0653 / 0.0075
plain_fedavg	0.992 / 0.994 / 0.990	0.013 / 0.002 / 0.092	0.0947 / 0.0301 / 0.0301
trimmed_mean_only	0.995 / 0.995 / 0.993	0.205 / 0.000 / 0.025	0.1070 / 0.0261 / 0.0299
autoencoder_only	0.105 / 0.073 / 0.051	0.105 / 0.073 / 0.051	0.0473 / 0.0411 / 0.0445
boosting_only	0.990 / 0.989 / 0.982	–	0.0001 / 0.0001 / 0.0003

TABLE X
POISONING SWEEP, FULL PIPELINE, 10 CLIENTS, 20 ROUNDS. SEEDS 42 / 43 / 44.

Attackers	Attack macro-recall	AE-alone recall	Survival	Benign FPR
0%	0.996 / 0.996 / 0.994	0.457 / 0.492 / 0.221	–	0.118 / 0.041 / 0.035
20%	0.995 / 0.996 / 0.994	0.466 / 0.483 / 0.258	0.30 / 0.40 / 0.23	0.105 / 0.094 / 0.037
30%	0.995 / 0.997 / 0.995	0.490 / 0.373 / 0.245	0.50 / 0.55 / 0.62	0.111 / 0.065 / 0.008
40%	0.994 / 0.994 / 0.992	0.297 / 0.139 / 0.074	0.60 / 0.64 / 0.61	0.091 / 0.038 / 0.043
50%	0.995 / 0.990 / 0.985	0.249 / 0.000 / 0.000	0.64 / 1.00 / 0.99	0.115 / 0.052 / 0.030

alarms vary. Compared with boosting alone at 30% attackers, the full pipeline gains 0.005 to 0.014 in macro-recall. Its packet false-alarm rate ranges from 0.75% to 11.1% across seeds, against at most 0.03% for the booster alone. The spread follows the long-connection effect described above.

C. Unseen attack types

Fig. 4 and Table XI report the leave-one-class-out experiment with no attackers. In all 42 cases (14 classes, three seeds), the cascade detects at least as much of the held-out class as the better of its two stages alone. Its mean detection rate over the 14 classes is 0.910, 0.944 and 0.895 for the three seeds, against 0.735, 0.757 and 0.723 for the classifier alone and 0.681, 0.728 and 0.590 for the autoencoder alone. The cost is a mean benign packet false-alarm rate of 0.079 to 0.104 across the held-out runs.

The held-out classes fall into three groups.

The classifier finds them anyway, under the wrong name. DDoS_ICMP, Fingerprinting, Port_Scanning and Ransomware are detected at 0.982 or more by the classifier alone, because it maps each to a behaviourally similar class it knows. Held-out Port_Scanning is labelled Fingerprinting in 99 to 100% of rows, and held-out Ransomware is labelled Backdoor in 97 to 99%. The detection counts, but an operator would see a confident alert with the wrong attack type.

Mostly the autoencoder finds them. For DDoS_UDP and MITM, the classifier alone detects 0.021 to 0.416 and 0.005 to 0.200. The cascade detects 0.803 to 0.905 and 0.751 to 1.000, almost all from the second stage. MITM is a single connection of 406 rows, so its numbers swing widely between seeds.

Hard for both, or seed-dependent. Uploading is the weakest class in every seed (cascade 0.527 to 0.665). The classifier lets 70 to 80% of it through as benign or with low confidence, and the autoencoder finds it only partly unusual. Backdoor (0.743 at seed 42, 0.998 otherwise) and Vulnerability_scanner (0.859 at seed 44, 0.992 otherwise) depend on the seed, in both cases through the classifier alone, which detects 0.486 and 0.335 in the weak seed against more than 0.89 in the others.

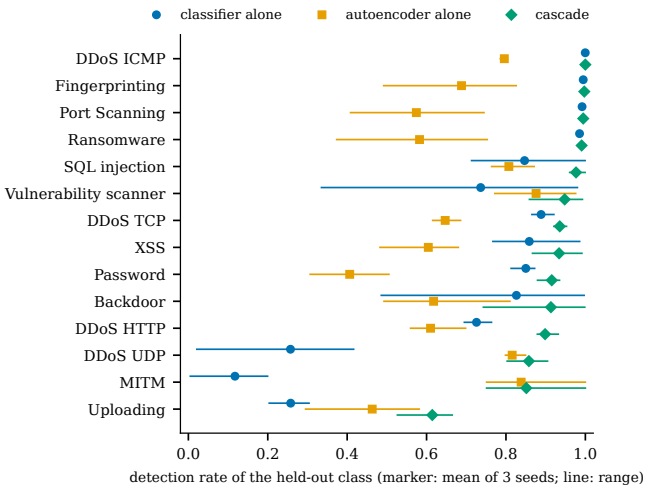


Fig. 4. Detection of each attack type when it is withheld from all training data, by stage. Classes are ordered by the cascade's mean detection rate.

D. An adaptive attacker

Table XII compares the stealth attacker with the sign-flipping one over three seeds. The filter catches the stealth attacker less often: it survives 0.60 to 0.95 of its rounds at 20% attackers, 0.68 to 0.83 at 30% and 0.80 to 0.85 at 40%, against 0.23 to 0.40, 0.50 to 0.62 and 0.60 to 0.64 for sign-flipping. The extra survival does not buy much more damage. The autoencoder-alone recall under stealth attackers is 0.220 to 0.588 at 20%, 0.316 to 0.427 at 30% and 0.045 to 0.334 at 40%; under sign-flipping it is 0.258 to 0.483, 0.245 to 0.490 and 0.074 to 0.297. Eight of the nine seed and fraction pairs differ by less than 0.1 between the two attackers, and the stealth attacker leaves the lower recall in five of the nine. Known-attack recall stays at 0.9908 or above in every run. We did not measure zero-day detection under either attacker, so the harm to the part of the system that matters most for unseen attacks is still open.

E. Pruning and the z-score clip

Table XIII reports pruning during federated training, over three seeds with no attackers. The autoencoder learns around the zeros up to a point. Its recall falls from 0.457, 0.492 and 0.221 at no pruning to 0.439, 0.376 and 0.210 at 50% sparsity, then to 0.350, 0.369 and 0.168 at 70%, 0.338, 0.209 and 0.145 at 80%, and 0.198, 0.059 and 0.092 at 90%. The

TABLE XI
LEAVE-ONE-CLASS-OUT DETECTION, 10 CLIENTS, 0% ATTACKERS, EVERY ROW OF THE HELD-OUT CLASS SCORED. SEEDS 42 / 43 / 44.

Held-out class	Rows	Classifier alone	Autoencoder alone	Cascade
Backdoor	24,167	0.486 / 0.996 / 0.997	0.493 / 0.550 / 0.810	0.743 / 0.998 / 0.998
DDoS_HTTP	49,768	0.695 / 0.764 / 0.719	0.572 / 0.698 / 0.560	0.879 / 0.932 / 0.885
DDoS_ICMP	68,758	1.000 / 1.000 / 0.999	0.786 / 0.802 / 0.800	1.000 / 1.000 / 1.000
DDoS_TCP	50,062	0.865 / 0.921 / 0.879	0.640 / 0.686 / 0.615	0.933 / 0.953 / 0.921
DDoS_UDP	121,567	0.416 / 0.021 / 0.334	0.849 / 0.799 / 0.799	0.905 / 0.803 / 0.866
Fingerprinting	872	0.995 / 0.994 / 0.994	0.826 / 0.747 / 0.492	0.998 / 0.998 / 0.997
MITM	406	0.148 / 0.200 / 0.005	0.764 / 1.000 / 0.751	0.803 / 1.000 / 0.751
Password	50,033	0.872 / 0.865 / 0.813	0.505 / 0.407 / 0.307	0.932 / 0.935 / 0.880
Port_Scanning	20,213	0.998 / 0.988 / 0.989	0.570 / 0.745 / 0.409	0.999 / 0.993 / 0.992
Ransomware	9,885	0.984 / 0.990 / 0.982	0.621 / 0.753 / 0.374	0.990 / 0.994 / 0.988
SQL_injection	50,981	0.828 / 0.713 / 0.999	0.764 / 0.871 / 0.787	0.961 / 0.970 / 0.999
Uploading	37,243	0.204 / 0.266 / 0.304	0.581 / 0.512 / 0.295	0.665 / 0.652 / 0.527
Vuln. scanner	50,044	0.980 / 0.894 / 0.335	0.880 / 0.976 / 0.772	0.992 / 0.993 / 0.859
XSS	15,172	0.823 / 0.986 / 0.767	0.680 / 0.650 / 0.483	0.943 / 0.991 / 0.867
Mean over 14 classes		0.735 / 0.757 / 0.723	0.681 / 0.728 / 0.590	0.910 / 0.944 / 0.895

TABLE XII
STEALTH VERSUS SIGN-FLIPPING ATTACKERS, 10 CLIENTS, 20 ROUNDS. EACH CELL LISTS SEEDS 42 / 43 / 44.

	Attacker survival		AE-alone recall		Known-attack recall
Attackers	Sign-flip	Stealth	Sign-flip	Stealth	Stealth
20%	0.30 / 0.40 / 0.23	0.60 / 0.70 / 0.95	0.466 / 0.483 / 0.258	0.454 / 0.588 / 0.220	0.9956 / 0.9961 / 0.9942
30%	0.50 / 0.55 / 0.62	0.73 / 0.68 / 0.83	0.490 / 0.373 / 0.245	0.427 / 0.329 / 0.316	0.9961 / 0.9956 / 0.9953
40%	0.60 / 0.64 / 0.61	0.80 / 0.80 / 0.85	0.297 / 0.139 / 0.074	0.334 / 0.160 / 0.045	0.9949 / 0.9939 / 0.9908

TABLE XIII
PRUNING DURING FEDERATED TRAINING (SERVER RE-PRUNES EACH ROUND), 10 CLIENTS, NO ATTACKERS. SEEDS 42 / 43 / 44.

Sparsity	Non-zero weights	AE-alone recall	Cascade macro-recall	Benign FPR
0%	9,600	0.457 / 0.492 / 0.221	0.996 / 0.996 / 0.994	0.118 / 0.041 / 0.035
50%	4,800	0.439 / 0.376 / 0.210	0.995 / 0.993 / 0.994	0.140 / 0.040 / 0.030
70%	2,880	0.350 / 0.369 / 0.168	0.993 / 0.996 / 0.994	0.120 / 0.078 / 0.024
80%	1,920	0.338 / 0.209 / 0.145	0.993 / 0.994 / 0.994	0.145 / 0.058 / 0.025
90%	960	0.198 / 0.059 / 0.092	0.993 / 0.994 / 0.993	0.115 / 0.026 / 0.025

TABLE XIV
PRUNING AFTER TRAINING (10-CLIENT, SEED-42 REAL-RUN BUNDLE, THRESHOLDS RECALIBRATED; ONE RUN). SPARSE SIZE IS RELATIVE TO THE DENSE MODEL.

Sparsity	Non-zero	Sparse size	AE recall	AE AUROC	Cascade macro-recall	Benign FPR
0%	9,600	1.98	0.461	0.892	0.996	0.1275
50%	4,800	1.00	0.375	0.788	0.995	0.0814
70%	2,880	0.61	0.098	0.495	0.994	0.0418
80%	1,920	0.41	0.110	0.451	0.995	0.0692
90%	960	0.22	0.092	0.461	0.995	0.0607
95%	480	0.12	0.093	0.460	0.995	0.0781
98%	192	0.06	0.094	0.467	0.995	0.0783

cascade's known-attack recall stays at 0.993 or above in every run, because the booster carries it.

Pruning after training is one run (the 10-client, seed-42 bundle, thresholds recalibrated; Table XIV) and is much worse at the same sparsity. Recall falls from 0.461 to 0.375 at 50% and to 0.098 at 70%, and the AUROC of the reconstruction error falls from 0.892 to 0.495, no better than chance. The 9,834-parameter autoencoder is already small, and a sparse encoding only saves space beyond 50% sparsity, where the model starts to lose recall. Pruning buys little here.

The z-score clip matters more than a single run suggested. Without it, the final benign validation loss of the autoencoder is 0.242 at seed 42, against 0.129 with the clip. At seeds 43 and 44 the loss without the clip is 117.8 and 102.8, against 0.057 and 0.264 with it. In two of three seeds, then, training without the clip fails outright, because rows from another part of the capture land thousands of standard deviations from a client's statistics and dominate its reconstruction error.

F. Live replay through the SDN

Phase B loaded the 10-client, seed-42 bundle and replayed each of the 14 attack captures from host h_1 (Table XV). Every replay ended in a block decision, and the rule `priority=100,ip,nw_src=10.0.0.1 actions=drop` was read back from the switch each time. Twelve of the 14 captures were given the right attack type, all by the booster. The MITM capture was called Ransomware, with only 542 of its 1,229 packets flagged. The OS-fingerprinting capture was called DDoS_ICMP. Both are still blocked. The replays took 8 to 268 s, the longest for the 3.2-million-packet DDoS_UDP capture.

Two facts limit what this shows. The captures are the dataset's own, so their rows are in the training data and the replay is a demonstration of the pipeline, not a held-out test. And the drop rule appears only after `tshark` has processed the whole capture: when it was read back, it had matched no packets, because the replay had already finished. Phase B is batch classification of a capture, not inline blocking.

TABLE XV
PHASE B ON EVERY ATTACK CAPTURE (10-CLIENT, SEED-42 BUNDLE). CONFIDENCE IS THE MEAN OVER THE NON-BENIGN PACKETS OF THE CHOSEN LABEL. ALL VERDICTS CAME FROM THE BOOSTER.

Capture	Classified as	Confidence	Packets flagged	Action	Wall time (s)
Backdoor attack	Backdoor	0.997	24,543 / 24,914	block	12
DDoS HTTP flood	DDoS_HTTP	0.966	119,748 / 229,142	block	29
DDoS ICMP flood	DDoS_ICMP	1.000	2,914,156 / 2,914,356	block	232
DDoS TCP SYN flood	DDoS_TCP	0.999	2,020,023 / 2,020,152	block	211
DDoS UDP flood	DDoS_UDP	0.999	2,899,758 / 3,215,732	block	268
MITM ARP spoofing + DNS	*Ransomware*	0.991	542 / 1,229	block	8
OS fingerprinting	*DDoS_ICMP*	0.980	1,134 / 1,176	block	9
Password attacks	Password	0.957	919,469 / 1,053,893	block	130
Port scanning	Port_Scanning	0.999	22,495 / 23,329	block	12
Ransomware	Ransomware	0.994	10,620 / 11,030	block	11
SQL injection	SQL_injection	1.000	51,097 / 51,228	block	17
Uploading	Uploading	0.999	37,592 / 37,644	block	15
XSS	XSS	0.999	15,677 / 16,215	block	12
Vulnerability scanner	Vulnerability_scanner	0.996	262,922 / 265,827	block	60

TABLE XVI
LIGHTGBM ON ALL ROWS, SEED 42, FROM THE BASE FEATURES TO THE FINAL 130. RANDOM = ROWS SPLIT AT RANDOM; GROUPED = CONNECTIONS HELD OUT. THE CEILING IS THE BEST ACCURACY AND MACRO RECALL ANY CLASSIFIER ON THAT FEATURE SET COULD REACH.

			Random		Grouped		Ceiling	
Features	Count	Rows	Acc.	F1	Acc.	F1	Acc.	Rec.
Base (authors' recipe)	91	1,909,719	0.9638	0.8868	0.9637	0.8892	0.9875	0.9692
+ service port	92	1,909,980	0.9802	0.9245	0.9803	0.9236	0.9985	0.9940
+ content	110	1,909,980	0.9802	0.9268	0.9802	0.9232	0.9985	0.9940
+ connection	130	1,985,768	0.9977	0.9879	0.9975	0.9808	0.9999	0.9997
+ class weights	130	1,985,768	0.9976	0.9867	0.9972	0.9828	0.9999	0.9997

G. Features for the first stage

Table XVI follows LightGBM from the 91 base features to the full 130 on all rows. On the connection-grouped split, macro F1 rises from 0.889 to 0.924 with the service port, stays level with the content features, and jumps to 0.981 with the connection features. Content features add little on their own, because few packets carry content, but they feed the connection aggregates. Class weights bring the final configuration to 0.983, with accuracy 0.997. The ambiguity ceiling rises from 0.969 to 0.9997 macro recall, and the share of rows whose feature vector also occurs with another label falls from 2.31% to 0.01%. The random and grouped splits differ by at most 0.007 in macro F1 for every feature set, so neither feature set was relying on memorised connections.

Fig. 5 shows where the gain comes from. The application-layer classes that the base features could not separate (Fingerprinting, Uploading, DDoS_HTTP, SQL_injection, Password and XSS) rise from 0.68–0.85 F1 to 0.93–1.00. Of the 1,113 remaining errors at seed 42, 54% are Password and DDoS_HTTP confused with each other, both HTTP requests to the same server, and 14% are Ransomware and Backdoor confused with each other (Fig. 6).

Table XVII repeats the final configuration over three seeds. Grouped macro F1 is 0.983, 0.984 and 0.985. Without class weights it swings from 0.981 to 0.952, because Fingerprinting, with few connections, varies widely. The same configuration trained on only 5% of the rows, the size of the calibration

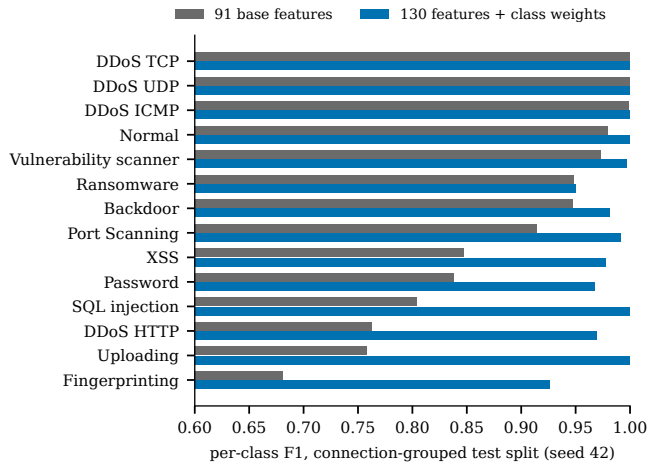


Fig. 5. Per-class F1 of LightGBM with the 91 base features and with the final 130 features and class weights, connection-grouped split, seed 42. MITM cannot be scored on this split.

set, reaches 0.997 accuracy and 0.979 macro F1 on the same grouped test set, against 0.962 and 0.881 for the base features under the same restriction. So the bootstrap booster in the federated system is not much weaker for training on the server's small set.

Two of the added features owe part of their value to the testbed. Benign traffic in Edge-IIoTset is MQTT and VNC and never HTTP, so a service port of 80 alone hints at an attack. In a network with benign web traffic, that feature would separate much less. The keyword groups were chosen with the dataset's attack families in mind, which makes them closer to signatures than to general behaviour.

Time-ordered split. Training on each class's earlier connections and testing on its later ones gives 0.994 accuracy and 0.912 to 0.913 macro F1 over three seeds (Table XVII). The lower macro F1 is mostly an artifact. MITM is one connection, so this split puts it entirely in the test side, where its recall of 0 enters the mean of 15 classes. Without MITM, the seed-42 macro F1 would be about 0.978. The real losses are on

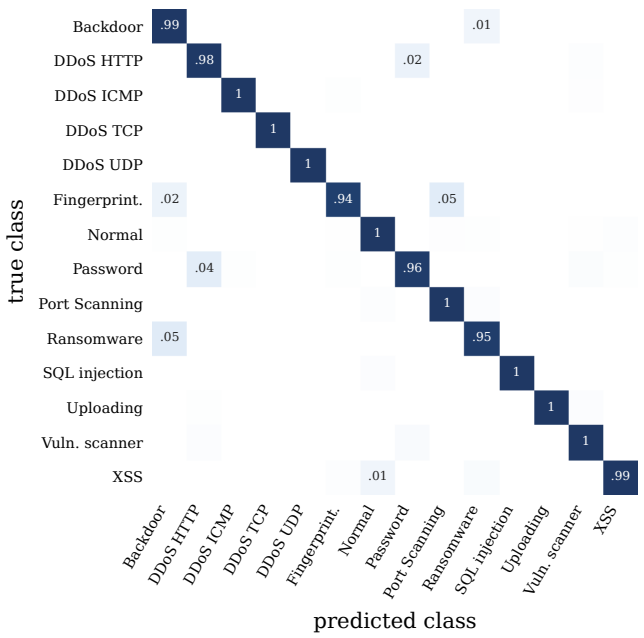


Fig. 6. Confusion matrix of the final classifier (130 features, class weights), connection-grouped test split, seed 42, row-normalised. Shading is stretched (square-root scale) so that small errors are visible.

TABLE XVII
FINAL CLASSIFIER CONFIGURATION ACROSS SEEDS: RANDOM SPLIT, GROUPED SPLIT, GROUPED SPLIT WITHOUT CLASS WEIGHTS, AND TIME-ORDERED SPLIT.

	Random	Grouped		Grouped, no weights	Time-ordered	
Seed	F1	Acc.	F1	F1	Acc.	F1
42	0.9867	0.9972	0.9828	0.9808	0.9942	0.9133
43	0.9868	0.9971	0.9835	0.9560	0.9943	0.9122
44	0.9866	0.9970	0.9851	0.9521	0.9941	0.9129

Backdoor (recall 0.947 to 0.962, against 0.989 on the grouped split) and DDoS_HTTP (0.912 to 0.925, against 0.980).

H. Communication, training time and decision time

The autoencoder's 9,834 float32 parameters cost $2 \times 9{,}834 \times 4 = 78{,}672$ bytes per client per round, downlink plus uplink: 786,720 bytes per round for 10 clients and 15,734,400 over 20 rounds. That figure leaves out the booster, which the server sends to every client every round. Its serialised file is 8.9 to 10.4 MB across the six runs, so one client's copy alone is more than ten times a whole round of autoencoder traffic for ten clients. Sending it only when its version changes, at most every third round, is a simple saving we have not implemented.

We timed the cascade on the saved 10-client, seed-42 bundle, one CPU thread of the VM's Intel Xeon, with 3,000 held-out test packets timed one call each and one batch of 200,000. The classifier alone takes a median 0.372 ms per packet (95th percentile 0.440), the autoencoder alone 0.060 ms (0.094), and both stages 0.642 ms (0.706). In batch mode the cascade handles 3,760 packets per second, 265.9 μs per packet, almost all of it in the booster (the autoencoder alone takes 5.5 μs). These are model times only: they exclude reading a capture, `tshark` feature extraction and the switch, and they come from one bundle on one machine, so we claim no confidence interval.

VII. Model Selection Study

The first stage of the cascade is LightGBM. This section gives the evidence behind that choice and says plainly where it is weaker than we first expected. The study ran in three stages: two earlier stages on other datasets that compared classical machine learning with deep learning, a wider survey of about 50 classical models on the same two datasets, and a new comparison on Edge-IIoTset that uses the split, features and metric of the rest of this paper. The earlier stages are public repositories of the same project; every number below is read from their result files.

A. Earlier stages on MQTTset and DataSense

Stage 1 compared eight supervised classifiers on MQTTset [39]: random forest, logistic regression, gradient boosting, k-nearest neighbours, naive Bayes, an MLP, a 1D-CNN and a GRU, plus a one-class autoencoder trained on benign traffic. An RBF SVM, an LSTM and a vanilla RNN were added to the table. Stage 2 repeated the method on DataSense [40], the 2025 sensor-based IIoT benchmark from the Canadian Institute for Cybersecurity. Table XVIII lists the results. Neither dataset gave a clear winner between the two families. On MQTTset, k-nearest neighbours had the highest macro F1 (0.819), ahead of the MLP (0.777), the GRU (0.768) and gradient boosting (0.724). On DataSense, the MLP (0.895) and gradient boosting (0.893) were level and the random forest followed at 0.884. The recurrent networks without sequence input were poor on both (LSTM 0.243 and vanilla RNN 0.095 macro F1 on DataSense). When the same CNN and GRU were retrained on windows of eight packets per flow, accuracy on MQTTset rose to 0.999, but that test measures a different task on a different sample and we do not compare it with the flat-feature models.

Stage 3 widened the classical side. It trained about 50 scikit-learn-compatible classifiers on both datasets, covering linear models, naive Bayes, single trees, bagged trees, boosting (XGBoost, LightGBM, CatBoost, HistGradientBoosting and others), kernel and instance-based learners, interpretable models and ensembles. It ranked them by a composite of quality (macro F1, weighted F1 and F1 on the rarest class) and cost (inference time and training time), with weights of 0.7 for quality and 0.3 for cost. Stage 3 reuses the split of the first two stages (a model in both gives the same macro F1; k-nearest neighbours with $k = 5$ scores 0.8188 on MQTTset in both), but it sub-sampled some slow models. Table XIX lists the ten highest composite scores. Gradient-boosting implementations take the first five places, XGBoost first, with LightGBM fourth. This ranking is the reason boosting went forward. It has two limits: the cost weights were our own choice, and both datasets are far smaller and simpler than Edge-IIoTset.

TABLE XVIII
EARLIER STAGES: CLASSICAL AND NEURAL CLASSIFIERS ON MQTTSET AND DATASENSE, ORDERED BY MACRO F1 WITHIN EACH DATASET. MQTTSET: 231,646 TRAINING AND 99,290 TEST ROWS, 33 FEATURES. DATASENSE: 31,538 TRAINING AND 13,517 TEST ROWS, 109 FEATURES. † TRAINED ON A 40,000-ROW SUBSAMPLE BECAUSE THE FULL FIT DID NOT FINISH.

Model	Accuracy	Macro F1	Train (s)
MQTTset			
K-Nearest Neighbors	0.9136	0.8188	0.0
MLP	0.8513	0.7768	42.8
GRU	0.8496	0.7681	122.7
Gradient Boosting	0.8883	0.7235	4.4
Random Forest	0.8856	0.7157	1.6
LSTM	0.8286	0.6560	276.8
Logistic Regression	0.6520	0.6305	9.7
1D-CNN	0.7312	0.6207	99.9
SVM (RBF)†	0.6467	0.5679	23.4
RNN	0.8202	0.5038	212.9
Naive Bayes	0.7946	0.4759	0.1
DataSense			
MLP	0.9448	0.8948	63.9
Gradient Boosting	0.9516	0.8931	7.0
Random Forest	0.9418	0.8842	0.8
1D-CNN	0.9244	0.8596	39.8
K-Nearest Neighbors	0.9129	0.8452	0.0
SVM (RBF)	0.8841	0.7667	22.2
GRU	0.8672	0.7470	126.5
Logistic Regression	0.8568	0.7357	1.3
Naive Bayes	0.4655	0.4344	0.0
LSTM	0.6228	0.2434	94.4
RNN	0.6072	0.0945	103.0

TABLE XIX
STAGE 3: TEN BEST OF 50 CLASSICAL MODELS BY AVERAGE COMPOSITE SCORE OVER MQTTSET AND DATASENSE (QUALITY WEIGHT 0.7, COST WEIGHT 0.3).

Model	Family	F1 MQTTset	F1 DataSense	Composite
XGBoost (DART)	boosting	0.8371	0.9618	99.9
XGBoost	boosting	0.8371	0.9618	99.9
Hist Gradient Boosting	boosting	0.8350	0.9585	99.5
LightGBM	boosting	0.8370	0.9614	99.5
CatBoost	boosting	0.8339	0.9523	99.2
Gradient Boosting (sklearn)	boosting	0.8334	0.9553	96.5
K-Nearest Neighbors (k=15, distance-weighted)	instance	0.8274	0.8487	92.4
Explainable Boosting Machine	interpretable	0.8284	0.9555	91.9
K-Nearest Neighbors (k=5)	instance	0.8188	0.8452	91.8
Voting Classifier (soft)	meta	0.7618	0.9210	90.5

B. Comparison on Edge-IIoTset

Protocol. The new comparison uses the 130-feature matrix and the connection-grouped split of Section VI-G (seed 42): 397,073 test rows, none from a connection seen in training. Every model trains on the same class-capped stratified sample of 500,000 training rows, because slow learners cannot be run on all 1.43 million, and is scored on the full test split. Hyperparameters were chosen on a separate validation sample of 60,000 rows and never on the test rows. Each model had a small grid (for example tree depth, number of leaves, regularisation strength or k); the grid and the chosen value are stored with the results. Models that accept class weights use the same square-root weights as the system, and the neural networks use them in the loss. The RBF SVM was trained on a 40,000-row subsample and the GRU on 150,000 rows, for the same feasibility reason. Metrics are accuracy, macro F1 over the classes present in the test split, benign false-positive rate, training time on 16 CPU threads and prediction time per row.

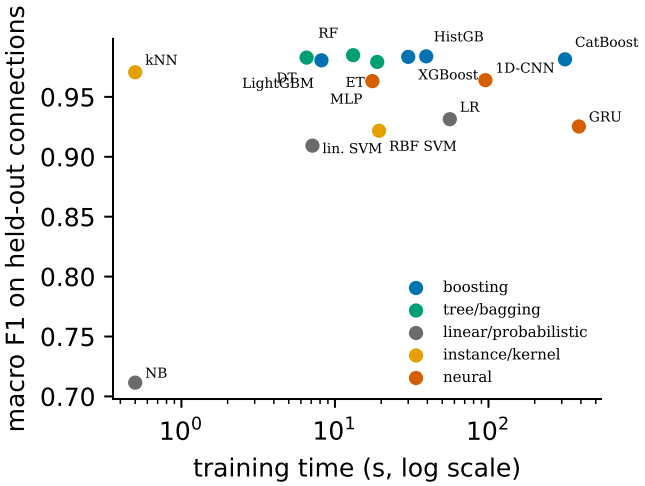


Fig. 7. Macro F1 against training time for the 15 classifiers of Table XX.

Results. Seven tree-based models fill the top of Table XX, with macro F1 between 0.979 and 0.985: random forest 0.985, HistGradientBoosting 0.984, XGBoost 0.983, a single decision tree 0.983, CatBoost 0.981, LightGBM 0.981 and extra trees 0.979. The spread of 0.006 is the same size as the seed-to-seed spread of one model (LightGBM on the full training set scores 0.983, 0.984 and 0.985 across the three seeds of Table XVII), so we do not rank these seven. k-nearest neighbours follows at 0.971, the 1D-CNN and the MLP at 0.964 and 0.963, logistic regression at 0.931, the RBF SVM on 40,000 rows at 0.922 and the linear SVM at 0.909. Gaussian naive Bayes collapses (0.712, with 59% of benign rows flagged); it was also among the weakest models in the earlier stages. The GRU, which reads the 130 features as a sequence of 130 steps, reached 0.925 after 20 epochs on 150,000 rows, but a first 3-epoch run scored only 0.106 and the 20-epoch validation score was still rising, so we treat it as under-trained and not as a final value for that architecture. The benign false-positive rate is 0.10 to 0.12% for every tree model and rises to 0.15 to 0.16% for the neural networks and logistic regression.

Two things follow. First, on this feature set the choice among tree models hardly matters, and a single decision tree of depth 20 is within 0.002 of the best. That fits the feature study of Section VI-G: once the connection features make the classes separable (the ambiguity ceiling is 0.9997 macro recall), the model has little left to add. Second, the trees beat the neural networks by about 0.02 macro F1 and the linear models by 0.05 or more. Cost does not separate the families as much as accuracy does: the trees train in 7 to 39 s (CatBoost 315 s), the MLP in 18 s, the 1D-CNN in 96 s and the GRU in 388 s, and a prediction takes 0.1 to 16 μs per row for every tree model, 0.2 to 26 μs for the networks, and 261 and 838 μs for the two instance-based and kernel models.

TABLE XX
EDGE-IIOTSET, 130 FEATURES, CONNECTION-GROUPED TEST SPLIT (397,073 ROWS, SEED 42): 15 CLASSIFIERS TRAINED ON THE SAME 500,000-ROW SAMPLE, ORDERED BY MACRO F1. TRAIN AND PREDICT TIMES ARE WALL-CLOCK ON 16 CPU THREADS, ONE JOB AT A TIME. VALUES MARKED ‡ COME FROM A SEPARATE RUN OF THAT MODEL ALONE ON THE MACHINE.

Model	Family	Acc.	Macro F1	Weighted F1	Benign FPR	Train (s)	Predict (μs/row)
Random forest	bagging trees	0.9970	0.9848	0.9970	0.0010	13	3.8
HistGradientBoosting	boosting	0.9970	0.9838	0.9970	0.0012	39‡	15.5‡
XGBoost	boosting	0.9971	0.9834	0.9972	0.0010	30	1.2
Decision tree	single tree	0.9967	0.9828	0.9968	0.0010	7	0.1
CatBoost	boosting	0.9970	0.9813	0.9970	0.0011	315	0.7
LightGBM	boosting	0.9971	0.9805	0.9971	0.0010	8	1.3
Extra trees	bagging trees	0.9968	0.9792	0.9968	0.0010	19	3.8
k-nearest neighbours	instance	0.9956	0.9706	0.9956	0.0012	0‡	261.3‡
1D-CNN	neural	0.9953	0.9640	0.9954	0.0016	96‡	17.6‡
MLP	neural	0.9959	0.9631	0.9959	0.0015	18‡	0.2‡
Logistic regression	linear	0.9937	0.9314	0.9941	0.0016	56	0.1
GRU (150k train rows)	neural	0.9857	0.9253	0.9856	0.0038	388	21.1
RBF SVM (40k train rows)	kernel	0.9901	0.9217	0.9905	0.0039	19‡	837.8‡
Linear SVM	linear	0.9912	0.9093	0.9925	0.0017	7	0.1
Gaussian naive Bayes	probabilistic	0.5258	0.7115	0.6456	0.5886	0	4.4

TABLE XXI
NINE CLASSIFIERS ON THE 91 BASE FEATURES AND ON THE 130 FEATURES (CONNECTION-GROUPED SPLIT, SEED 42; 500,000 TRAINING ROWS). THE LAST COLUMN IS THE MACRO-F1 GAIN FROM THE 39 ADDED FEATURES.

Model	Acc. (91)	Macro F1 (91)	Macro F1 (130)	Gain
XGBoost	0.9416	0.8748	0.9834	+0.1086
LightGBM	0.9330	0.8541	0.9805	+0.1264
CatBoost	0.9311	0.8519	0.9813	+0.1294
Random forest	0.9104	0.8068	0.9848	+0.1780
Extra trees	0.9056	0.7877	0.9792	+0.1914
Decision tree	0.8912	0.7774	0.9828	+0.2054
MLP	0.8240	0.5956	0.9631	+0.3675
k-nearest neighbours	0.7349	0.5603	0.9706	+0.4103
Logistic regression	0.4781	0.4018	0.9314	+0.5296

TABLE XXII
LIGHTGBM AGAINST A DENSE MLP ON THE 91 BASE FEATURES, 500,000-ROW CLASS-CAPPED SAMPLE (400,000 TRAINING AND 100,000 TEST ROWS, RANDOM SPLIT; NORMAL IS ABOUT 10% OF THE SAMPLE). THE MLP HAS HIDDEN LAYERS OF 64 AND 32 UNITS, DROPOUT 0.1 AND 8,271 PARAMETERS, TRAINED FOR 15 EPOCHS.

	LightGBM	MLP
Accuracy	0.8908	0.6779
Macro F1	0.8872	0.6218
Weighted F1	0.8924	0.6681
Benign FPR	0.1599	0.5341
Training time (s)	55.5	10.2
Model size	12.8 MB (300 rounds)	8,271 parameters

C. Boosting on the base features

The earlier head-to-head of LightGBM and an MLP used the 91 base features and showed a large gap, 0.887 against 0.622 macro F1 on a 500,000-row class-capped sample (Table XXII). The gap was largest on the application-layer attacks that the base features cannot separate. To see how much of it comes from the features rather than the learners, we repeated the comparison above with the 91 base features. Nine of the 15 models were run (the other six were not repeated); Table XXI gives the results. On the base features the boosting implementations lead clearly: XGBoost 0.875, LightGBM 0.854 and CatBoost 0.852, against 0.807 for the random forest, 0.788 for extra trees and 0.777 for a single decision tree. The MLP (0.596), k-nearest neighbours (0.560) and logistic regression (0.402) are far behind. The MLP's score is close to the 0.622 of the earlier head-to-head, which used a different sample. Adding the 39 service-port, content and connection features raises every model, from $+0.11$ for XGBoost to $+0.53$ for logistic regression, and it closes the gap between boosting and the bagged trees: random forest gains $+0.18$ and reaches the top of Table XX. The two feature sets give different test rows (381,742 and 397,073, because deduplication differs), so the differences are approximate.

So the earlier result and the new one agree. Boosting is the best choice when the features are weak, and its edge over bagged trees disappears when the features are strong.

In that comparison the gap is largest on XSS (recall 0.761 against 0.072), SQL_injection (0.767 against 0.213), Fingerprinting (0.579 against 0.000), Port_Scanning (0.997 against 0.499) and Uploading (0.630 against 0.198), and near zero on the three DDoS floods, which both models classify almost perfectly. The MLP is slightly better on Password and DDoS_ICMP. The comparison is limited: the MLP is small and was not tuned, the sample uses the unrepaired MITM rows, and its class mix makes the false-alarm rates incomparable with the rest of the paper.

D. Why LightGBM

Three results favour boosting and one does not. Boosting implementations took the first five places among 50 classical models on two other datasets (stage 3), and on the 91 base features LightGBM beat the MLP by 0.27 macro F1. On the 130 features LightGBM sits inside the top group of tree models, trains in 8 s and produces a model of 9 to 10 MB. It also supports continued training from an existing model, which the incremental update of Section IV-B needs. What the study

does not show is that boosting is the best first stage on the 130 features: random forest, HistGradientBoosting, XGBoost and a single decision tree scored as high or higher in this run. We kept LightGBM because the system was built and evaluated around it before this comparison was run.

VIII. Discussion

A. What each part contributes

Measured separately, the five parts do different jobs. The booster detects almost all known attacks (micro-recall 0.9954 to 0.9981 in the real runs). Because it is trained on the server, poisoned updates cannot reach it. The federated autoencoder adds at most 0.0008 on known attacks and supplies most of the system's benign false alarms. It earns its place on attack types the booster has never seen, where the cascade raises mean detection from 0.723–0.757 to 0.895–0.944, and on classes such as DDoS_UDP and MITM it is nearly the whole defence. The trust filter, with the norm clip and trimmed mean, keeps the autoencoder usable at 30% sign-flipping attackers, where plain FedAvg destroys it. SDN enforcement works end to end on all 14 captures, but only as batch classification of a finished capture.

A cascade-level recall therefore says little about whether the federated part works. With 30% attackers, plain FedAvg keeps 0.990 to 0.994 known-attack macro-recall while its autoencoder detects between 0.2% and 9.2% of attack packets. We think evaluations of hybrid federated detectors should report the federated component's own recall and its detection of unseen classes, not only the combined number.

B. False alarms

The packet false-alarm rate (2.5 to 16.3%) would make the system unusable if it were the right measure. It is driven by a few very long benign connections. When one of them looks unusual to a client's autoencoder, thousands of packets are flagged at once. Per connection the rate is 0.34 to 1.04%, and the median client sits near the 3% the threshold implies. Two changes would address this directly. A device verdict could require a minimum share of flagged packets within a time window, rather than the current rule of any one non-benign packet. And the threshold could be set per connection rather than per packet. Neither is implemented.

C. The trust filter at scale

At 30% attackers the filter admits attackers in 50 to 62% of their rounds, and the stealth attacker passes even more often. The autoencoder still holds, which suggests that the norm clip and trimmed mean after the filter do much of the work, although we have not isolated their share. Even with no attackers, it excludes honest clients in 3 to 6% of client-rounds. Both observations come from the same root. With non-IID data, honest clients' updates differ from the median by amounts that overlap those of attackers, a problem documented across robust aggregation rules [6]. A client whose local data is poisoned, but which trains honestly on it, cannot be separated from an honest heterogeneous client by any test on update direction. The server already holds a labelled calibration set. Using it to compute a reference update, as FLTrust does with its root dataset [5], is the most direct improvement available, and we have not tested it. We have not established why the autoencoder is weak when used alone in some seeds (0.220 at 10 clients, seed 44). Candidates include the connection-grouped benign thresholds and the larger, more homogeneous local data at full scale.

D. How federated is the system?

Only the autoencoder is trained federatedly. The server holds about 100,000 labelled rows and trains the booster on 71,853 to 79,866 of them, and the incremental update moves up to 50 confirmed alert rows per surviving client to the server every three rounds. We chose this because labelled attack traffic is rare at the edge and a supervised first stage needs labels from somewhere. The feature study shows that the small set is enough: trained on 5% of the rows, the final classifier reaches 0.979 macro F1 on held-out connections, against 0.983 with 72%. A deployment that cannot accept any central labelled set would need federated boosting [27], which we have not implemented.

E. Limitations and threats to validity

1) **One dataset for the system.** All system results are on Edge-IIoTset. The model-selection study includes two other datasets, but we have not tested whether the corrections, the filter's behaviour or the cascade's gains carry over to them.
2) **Replays are not held-out tests.** Phase B replays the dataset's own captures. It classifies a whole capture before acting, so it does not block inline, and we report no mitigation latency.
3) **Three seeds at most.** The ablation, sweep, zero-day experiment and real runs have three seeds, which is too few for confidence intervals. Several measures vary widely between seeds, the autoencoder's recall above all. Pruning after training is a single run.
4) **Two attacker models.** We test sign-flipping and one white-box strategy. Other adaptive attacks [21] are untested.
5) **Filter limits.** The filter admits many attackers at 20 to 40%, fails at 50%, and excludes honest non-IID clients. The trimmed mean alone cannot absorb more colluding attackers than it trims.
6) **False alarms.** The packet rate is high and concentrated in long connections. The device verdict fires on any single non-benign packet, so a benign device with enough traffic would almost certainly be flagged. We have not evaluated Phase B on benign captures.
7) **MITM.** MITM is a single connection, so it cannot be split. It is never in a test slice and is scored only by the zero-day experiment and the replay, where it is blocked but misnamed.
8) **Clients without benign validation data** get an infinite threshold, so their autoencoder never flags. Up to three clients per run were affected.

9) **Testbed-specific features.** The service port and the keyword groups benefit from how Edge-IIoTset was built. Connection features summarise records of up to 120 s, so the system labels connection records, not single packets as they arrive.
10) **Privacy relaxation.** The incremental booster update sends confirmed alert rows to the server.
11) **In-process evaluation.** The ablation, sweep and zero-day experiments run in one process without network effects. The real runs agree with them closely (Section VI).
12) **Communication cost** excludes the booster broadcast, which is much larger than the autoencoder traffic.
13) **Comparability.** Because we correct the dataset and split by connection, our numbers are not directly comparable with results on the released CSV. Given the leak, we would expect those results to be inflated, but we have not re-run other published models to show it.
14) **Time order.** The CSV has no dates, so the time-ordered split orders rows by position, and no split by capture file was possible.
15) **Novelty.** Our search for systems that combine all five parts was targeted, not systematic.

IX. Conclusion and Future Work

FL-IDS combines a LightGBM first stage, a federated autoencoder, an update-level trust filter and OpenFlow enforcement. We trained and tested it on all 1,985,768 usable rows of Edge-IIoTset, with connections kept whole across every split, after correcting three defects in the released data. Known attacks are detected almost completely, mostly by the booster. The federated autoencoder raises detection of attack types withheld from training from 0.723–0.757 to 0.895–0.944 on average, and the trust filter keeps it working at 30% sign-flipping attackers where plain FedAvg does not. The same results show where the system falls short. At this scale the filter admits many attackers, it fails at 50%, and an attacker that knows its rules passes most of the time. Packet-level false alarms are high, though they are concentrated in a few long connections and fall to about 1% when counted per connection.

The next steps follow from these limits:

- use the server's calibration set as a reference direction for the filter, and measure zero-day detection under the stealth attacker and for pruned models;
- replace the any-packet device verdict with a windowed, connection-level rule, move to streaming classification, and measure detection-to-rule latency on held-out and benign traffic;
- broadcast the booster only when it changes, and evaluate federated boosting so that no labelled data needs to be held centrally;
- test the system on a second dataset.

Acknowledgements

We thank our project guide, Mr. Syed Akram, Department of Computer Science & Engineering, BMS College of Engineering, for his guidance throughout this work.

Code and Data Availability

The implementation, configurations, result files and test suite are at https://github.com/Aronnnn1/FL-IDS. Edge-IIoTset is available from its authors [14]. The repaired MITM rows can be regenerated from the authors' capture with the repository's repair script.

References

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist. (AISTATS)*, ser. Proc. Mach. Learn. Res., vol. 54, 2017, pp. 1273–1282.
[2] J. L. Hernández-Ramos, G. Karopoulos, E. Chatzoglou, V. Kouliaridis *et al.*, "Intrusion detection based on federated learning: A systematic review," *ACM Comput. Surv.*, vol. 57, no. 12, 2025.
[3] P. Blanchard, E. M. E. Mhamdi, R. Guerraoui, and J. Stainer, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems 30*, 2017.
[4] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., vol. 80, 2018, pp. 5650–5659.
[5] X. Cao, M. Fang, J. Liu, and N. Z. Gong, "FLTrust: Byzantine-robust federated learning via trust bootstrapping," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2021.
[6] S. Li, E. C.-H. Ngai, and T. Voigt, "An experimental study of Byzantine-robust aggregation schemes in federated learning," *IEEE Trans. Big Data*, vol. 10, no. 6, pp. 975–988, 2024.
[7] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar *et al.*, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 38, no. 2, pp. 69–74, 2008.
[8] M. M. Galal, "Provenance, not behaviour: A serialisation artifact in Edge-IIoTset and a leakage-free benchmark for precision-agriculture intrusion detection," arXiv:2608.15761, 2026.
[9] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Secur. Symp.*, 2022, pp. 3971–3988.
[10] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, 2021, pp. 7–12.
[11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017.
[12] H. Zhang, L. Ge, G. Zhang, J. Fan *et al.*, "A two-stage intrusion detection method based on light gradient boosting machine and autoencoder," *Math. Biosci. Eng.*, vol. 20, no. 4, pp. 6966–6992, 2023.
[13] D. J. Beutel, T. Topal, A. Mathur, X. Qiu *et al.*, "Flower: A friendly federated learning research framework," arXiv:2007.14390, 2020.
[14] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40 281–40 306, 2022.
[15] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh *et al.*, "Federated-learning-based anomaly detection for IoT security attacks," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2545–2554, 2022.
[16] B. Li, Y. Wu, J. Song, R. Lu *et al.*, "DeepFed: Federated deep learning for intrusion detection in industrial cyber–physical systems," *IEEE Trans. Ind. Informat.*, vol. 17, no. 8, pp. 5615–5624, 2021.
[17] V. Rey, P. M. S. Sánchez, A. H. Celdrán, and G. Bovet, "Federated learning for malware detection in IoT devices," *Comput. Netw.*, vol. 204, p. 108693, 2022.
[18] V. T. Nguyen and R. Beuran, "FedMSE: Semi-supervised federated learning approach for IoT network intrusion detection," *Comput. Secur.*, vol. 151, p. 104337, 2025.
[19] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp. (NDSS)*, 2018.
[20] Z. Y. Sazid and R. Abbas, "Cybersecurity in edge computing: A trust-aware federated hybrid intrusion detection framework," arXiv:2609.39584, 2026.
[21] M. Fang, X. Cao, J. Jia, and N. Gong, "Local model poisoning attacks to Byzantine-robust federated learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1623–1640.

[22] J. Xu, S.-L. Huang, L. Song, and T. Lan, "Byzantine-robust federated learning through collaborative malicious gradient filtering," in *Proc. IEEE 42nd Int. Conf. Distrib. Comput. Syst. (ICDCS)*, 2022, pp. 1223–1235.
[23] Z. Li, L. Liu, J. Zhang, and J. Liu, "Byzantine-robust federated learning through spatial-temporal analysis of local model updates," in *Proc. IEEE 27th Int. Conf. Parallel Distrib. Syst. (ICPADS)*, 2021, pp. 372–379.
[24] C. Fung, C. J. M. Yoon, and I. Beschastnikh, "The limitations of federated learning in sybil settings," in *Proc. 23rd Int. Symp. Res. Attacks, Intrusions Defenses (RAID)*, 2020, pp. 301–316.
[25] T. Zhu, Z. Guo, C. Yao, J. Tan *et al.*, "Byzantine-robust federated learning via cosine similarity aggregation," *Comput. Netw.*, vol. 254, p. 110730, 2024.
[26] T. T. Thein, Y. Shiraishi, and M. Morii, "Personalized federated learning-based intrusion detection system: Poisoning attack and defense," *Future Gener. Comput. Syst.*, vol. 153, pp. 182–192, 2024.
[27] K. Cheng, T. Fan, Y. Jin, Y. Liu, T. Chen, D. Papadopoulos, and Q. Yang, "SecureBoost: A lossless federated learning framework," *IEEE Intell. Syst.*, vol. 36, no. 6, pp. 87–98, 2021.
[28] C. Thapa, K. K. Karmakar, A. H. Celdrán, S. Camtepe *et al.*, "FedDICE: A ransomware spread detection in a distributed integrated clinical environment using federated learning and SDN based mitigation," in *Quality, Reliability, Security and Robustness in Heterogeneous Systems (QShine 2021)*, ser. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering, vol. 402. Springer, 2021, pp. 3–24.
[29] Z. A. E. Houda, A. S. Hafid, and L. Khoukhi, "MiTFed: A privacy preserving collaborative network attack mitigation framework based on federated learning using SDN and blockchain," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 4, pp. 1985–2001, 2023.
[30] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, 2010, pp. 305–316.
[31] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Secur. Privacy (ICISSP)*, 2018, pp. 108–116.
[32] E. Breck, N. Polyzotis, S. Roy, S. E. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. Conf. Syst. Mach. Learn. (SysML)*, 2019.
[33] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. 3rd Int. Conf. Learn. Represent. (ICLR)*, 2015.
[34] C. Leys, C. Ley, O. Klein, P. Bernard, and L. Licata, "Detecting outliers: Do not use standard deviation around the mean, use absolute deviation around the median," *J. Exp. Soc. Psychol.*, vol. 49, no. 4, pp. 764–766, 2013.
[35] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. 9th ACM SIGCOMM Workshop Hot Topics Netw. (HotNets)*, 2010, pp. 1–6.
[36] B. Pfaff, J. Pettit, T. Koponen, E. Jackson *et al.*, "The design and implementation of Open vSwitch," in *Proc. 12th USENIX Symp. Netw. Syst. Design Implement. (NSDI)*, 2015, pp. 117–130.
[37] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," arXiv:1909.06335, 2019.
[38] A. Paszke, S. Gross, F. Massa, A. Lerer *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, 2019.
[39] I. Vaccari, G. Chiola, M. Aiello, M. Mongelli *et al.*, "MQTTset, a new dataset for machine learning techniques on MQTT," *Sensors*, vol. 20, no. 22, p. 6578, 2020.
[40] A. Firouzi, S. Dadkhah, S. Maret, and A. A. Ghorbani, "DataSense: A real-time sensor-based benchmark dataset for attack analysis in IIoT with multi-objective feature selection," *Electronics*, vol. 14, no. 20, p. 4095, 2025.